

UCB CS294/194-196: Agentic AI (Fall 2025)

Codex based on official course pages, slides, readings, subtitles, and lecture videos

2026 年 4 月 16 日

目录

课程说明	2
来源说明	2
课程目录	2
1 Introduction	2
2 LLM Agents Overview	11
3 Evolution of system designs from an AI engineer perspective	22
4 Post-Training Verifiable Agents	31
5 Agent Evaluation & Project Overview	38
6 Some Challenges and Lessons from Training Agentic Models	46
7 Multi-Agent AI	53
8 Predictable Noise in LLM	63
9 AI Agents to Automate Scientific Discoveries	72
10 Practical Lessons from Deploying Real-World AI Agents	80
11 Multi-Agent Systems in the Era of LLMs Oriol Vinyals,	87
12 Autonomous Agents: Embodiment, Interaction, and Learning	94
13 Agentic AI Safety & Security	101

课程说明

本讲义先逐讲生成，再按播放列表顺序合并。

来源说明

主课程来源为 Berkeley Fall 2025 官方课程页 (course page) 与公开 MOOC syllabus。公开视频主入口为 Agentic AI MOOC Fall 2025 playlist。本书在必要处显式参考以下补充课程：

- UCB CS294/194-196: Large Language Model Agents (Fall 2024): official page
- 该补充课程的公开视频入口：playlist
- Stanford CS329A: Self-Improving AI Agents (Autumn 2025): official page

以下讲次存在官方 source gap，但已按 best effort 规则补写：

- 2025-09-08: Introduction。
- 2025-11-24: No lecture —Thanksgiving week。
- unknown: Agent Evaluation & Project Overview。

课程目录

讲次	主题	日期
01	Introduction	2025-09-08
02	LLM Agents Overview	2025-09-15
03	Evolution of system designs from an AI engineer perspective	2025-09-22
04	Post-Training Verifiable Agents	2025-09-29
05	Agent Evaluation & Project Overview	2025-10-06
06	Some Challenges and Lessons from Training Agentic Models	2025-10-13
07	Multi-Agent AI	2025-10-20
08	Predictable Noise in LLM	2025-10-27
09	AI Agents to Automate Scientific Discoveries	2025-11-03
10	Practical Lessons from Deploying Real-World AI Agents	2025-11-10
11	Multi-Agent Systems in the Era of LLMs Oriol Vinyals,	2025-11-17
12	Autonomous Agents: Embodiment, Interaction, and Learning	2025-12-01
13	Agentic AI Safety & Security	2025-12-08

1 Introduction

课程讲义

CS294/194-196 第 1 讲：课程导论与 Agentic AI 全景

Codex 基于 UCB Agentic AI Fall 2025 官方 slides、课程页与公开 syllabus 重写

2026-04-16

课程：UCB CS294/194-196: Agentic AI (Fall 2025)

讲次：导论 (Introduction)

主讲：Dawn Song

来源：官方课程页与导论 slides

说明：本讲官方课程页未提供公开视频，因此本章依据官方课件与课程页信息做 best effort 教材化重建。

目录

1	为什么要单独开一门 Agentic AI 课程	2
1.1	本章小结	2
2	Agentic AI 的最小框架：工具、记忆、规划与环境交互	2
2.1	为什么课程把 Retrieval 也放进 agent 框架	3
2.2	本章小结	3
3	为什么 2025 被课程直接称作 “Year of Agents”	4
3.1	从 competitive math / coding 到真实环境任务	4
3.2	应用扩张与系统复杂度同时上升	4
3.3	本章小结	4
4	这门课到底覆盖什么：从 reasoning 到 robotics 的完整技术栈	4
4.1	第一层：基础认知与任务推理	5
4.2	第二层：能力扩展接口	5
4.3	第三层：系统化场景	5
4.4	第四层：治理与可靠性	5
4.5	本章小结	6
5	课程组织与学习要求：为什么这是 “做项目” 的课，而不只是 “听 guest talk” 的课	6
5.1	每次 lecture 后的 quiz 有什么作用	6
5.2	Semester-long project 是这门课的核心训练方式	6
5.3	对自学者的等价要求	6
5.4	本章小结	6
6	总结与延伸	7

1 为什么要单独开一门 Agentic AI 课程

这份导论没有从某个单点技术出发，而是先确立整门课的范围与判断：**Agentic AI 已经不再只是“给 LLM 加几个工具调用”的工程技巧，而是在 2025 年被视为一类足以重塑 AI 应用与系统栈的主线范式**。课程页与导论 slides 明确把它放在 LLM foundations、reasoning、planning、tool use、memory、robotics、web automation、scientific discovery、safety & security 这些主题的交叉点上。

第 1 讲的教学目标因此不是立刻展开公式推导，而是回答三个更根本的问题：

1. 什么叫 agentic AI，它与传统 chatbot 或单轮 LLM 调用究竟差在哪？
2. 为什么 2025 年会被课程直接称作“Year of Agents”？
3. 如果这门课要做教材级学习，后面 12 次课分别在整条技术链上承担什么角色？

导论章的定位

本章承担的是“定义战场”的任务。后面各讲会进入 reasoning、evaluation、multi-agent、scientific discovery、robotics 和 safety 等细节，但如果没有这一讲先给出统一的 agent 框架，整门课很容易被误读成若干 guest lecture 的拼盘。

1.1 本章小结

导论首先把 Agentic AI 从一个模糊流行词收束成课程对象：它关心的是 LLM 如何在记忆、工具、反馈、规划和环境交互中形成更完整的任务执行闭环，而不是只生成一段高质量文本。

2 Agentic AI 的最小框架：工具、记忆、规划与环境交互

导论 slides 给出的核心结构图非常简练，但已经足以作为整门课的统一坐标系：LLM 位于中心，外圈围绕着 **tool use**、**retrieval**、**memory**、**reasoning & planning**、**feedback** 和 **environment**。这张图的价值在于，它把 agent 定义成一个闭环系统，而不是一个只在静态 prompt 上做条件生成的模型。

LLM agents: enabling LLMs to interact with the environment

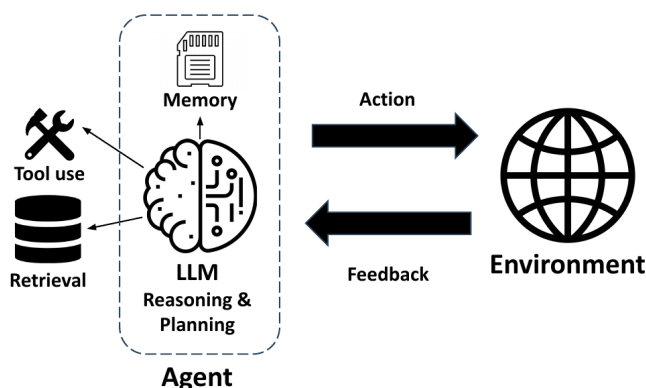


图 1: LLM agent 的基本闭环: memory、tool use、retrieval、reasoning & planning、action 与 environment feedback 共同构成智能体。

如果把它写成更适合教材阅读的语言, 可以把 Agentic AI 的最小 workflow 概括成:

1. 接收任务与环境状态;
2. 通过内部推理与规划形成若干候选步骤;
3. 调用工具、检索知识或访问外部接口;
4. 根据环境反馈修正记忆与后续动作;
5. 迭代直到完成任务或触发停止条件。

这一定义有两个含义。第一, agent 的能力并不只来自 base model 的参数规模, 也来自外部结构化接口。第二, 真实世界任务通常不是“一次性算出来”, 而是要经历试错、回退、重规划与跨步骤记忆维护。也正因为如此, 课程会把 planning、memory、tool use、evaluation 和 safety 全部并入主线。

为什么课程图里 LLM 不再是终点

在普通 chatbot 视角下, LLM 的输出就是系统终点; 在 agent 视角下, LLM 的输出更像一个中间决策节点。真正闭环的是“模型输出动作, 再从环境拿回反馈, 再继续做下一步决策”。

2.1 为什么课程把 Retrieval 也放进 agent 框架

导论图里单独画出了 retrieval, 这说明课程不把检索当作附属技巧。它的作用至少有三层: 一是为 agent 提供超出 parametric memory 的外部知识; 二是降低纯内部推理的幻觉风险; 三是让系统能够在长任务里不断补充任务相关上下文。换句话说, retrieval 在这里不是 RAG 的老问题重提, 而是 agent 扩展工作记忆和外部知识边界的关键接口。

2.2 本章小结

从这张框架图出发, 整门课的后续结构就都变得自然了: reasoning 与 planning 解决“怎么想”, tool use 与 retrieval 解决“怎么做”, memory 解决“怎么跨步骤保持状态”, feedback 与 environment

解决“怎样形成闭环”。

3 为什么 2025 被课程直接称作“Year of Agents”

导论 slides 很明确地做了一个时间判断：Fall 2024 开 Berkeley LLM Agents MOOC 时，业界对 agents 的集中讨论还远没有这么热；而到 2025 年，课程直接把这一年标记为“Year of Agents”。这并不是一句宣传口号，而是在给学生一个观察框架：**某种技术从实验室概念变成产业与研究共同追逐的主线时，会同时在应用层、系统层和 benchmark 层出现密集信号。**

导论给出的这些信号主要体现在两个方面。第一，agent 已经快速渗透到多类应用：code generation、computer use、personal assistants、robotics，以及教育、法律、金融、医疗、网络安全等垂直场景。第二，frontier AI 的进展开始越来越频繁地以“自主完成复杂任务”的形式被展示，而不只是静态文本生成质量。

3.1 从 competitive math / coding 到真实环境任务

课件里把 competitive math 和 coding 的突破单独拿出来，是因为这两类任务天然检验多步推理、搜索、验证与错误修正能力。它们的信号价值在于：模型不再只是“看起来像会做题”，而是在更严格的任务标准上逼近甚至达到高水平人类表现。导论并没有因此声称“agent 已经成熟”，相反，它是在说明为什么 reasoning 和 evaluation 会在后续课程里占据核心位置。

3.2 应用扩张与系统复杂度同时上升

一旦 agent 落到 code generation、computer use、robotics 和 scientific discovery 这些场景，系统要求就会比普通聊天更严苛。因为这些任务不仅要求正确回答，还要求：

- 维护长时间尺度上的状态一致性；
- 与外部工具、浏览器、代码仓库、仿真器或机器人接口对接；
- 在失败时能回滚、重试或重新规划；
- 有可评估、可复现、可审计的任务结果。

这也解释了为什么课程后面不仅有 reasoning / planning，也有 infrastructure、evaluation，以及 safety & security。导论实际上已经提前告诉读者：**Agentic AI 从一开始就是系统问题，不只是模型问题。**

3.3 本章小结

“Year of Agents”在这门课里的真实含义，是 agent 已经从一个外围话题变成组织前沿 AI 能力的主线视角。后续课程因此不会只讲单个模型技巧，而会围绕任务闭环、系统栈和真实应用来展开。

4 这门课到底覆盖什么：从 reasoning 到 robotics 的完整技术栈

导论 slides 直接给出了一组代表性主题：fundamental reasoning techniques、tool using in agents、agents for software engineering、agents stack & infrastructure、agentic workflows / real-world applications、safety and security。如果只把这些主题列出来，它们像课程广告；但把它们按教材逻辑重新组织后，会发现其实是四层结构。

Sample Topics covered in this course

- Fundamental reasoning techniques
- Tool using in agents
- Agents for software engineering
- Agents Stack & Infrastructure
- Agentic workflows, real-world applications
- Safety and security

图 2: 导论 slides 直接给出的课程主题分布: 从 reasoning、tool use 到 software engineering、infrastructure 与 safety。

4.1 第一层: 基础认知与任务推理

第一层是 reasoning、planning、verification、evaluation。这一层解决的是 agent 如何在多步任务里做出更可靠的中间决策, 以及我们怎么判断这种决策是否真的有效。没有这一层, agent 就只剩“会调用工具”的表面形式。

4.2 第二层: 能力扩展接口

第二层是 tool use、retrieval、memory 和 external feedback。它们回答的是 agent 如何突破单模型静态上下文的限制, 获得更强的任务执行能力。课程把这些内容纳入主线, 说明它关心的是完整执行系统, 而不是只谈 base model。

4.3 第三层: 系统化场景

第三层是 software engineering、computer use、scientific discovery、robotics 等高价值场景。导论里点名这些例子, 等于是提前告诉学生: 后续讲次不会停留在抽象讨论, 而会不断把 agent 放回真实应用 workflow。

4.4 第四层: 治理与可靠性

最后一层是 evaluation、safety & security。它们不是课程的尾声附录, 而是与前面几层并列的核心问题。原因非常直接: agent 的能力越接近真实任务执行, 其失误和滥用的代价就越高, 因此安全性与可评估性必须从框架层一起设计。

为什么这门课不能只讲“更强的 agent”

如果只追求更强任务完成率, 而不讲 evaluation 与 safety, 那么课的结尾就会失真。因为现实中的 agent 不是 benchmark 上的抽象智能体, 而是会接入工具、执行代码、操纵浏览器甚至作用于物理世界的系统。

4.5 本章小结

导论给出的主题列表，本质上是一张分层地图：底层是 reasoning/planning，中层是 memory/-tool/retrieval，上层是 code/robotics/science 等应用，再往外一圈是 evaluation 与 safety。后面逐讲展开时，都可以在这张地图上找到位置。

5 课程组织与学习要求：为什么这是“做项目”的课，而不只是“听 guest talk”的课

导论后半段专门讲课程安排、quiz、semester-long project 和推荐背景。这部分对教材读者同样重要，因为它说明授课团队希望学生获得的不只是概念理解，而是能真正做出 agentic AI 系统。

5.1 每次 lecture 后的 quiz 有什么作用

课件明确写了 quizzes after each lecture，并规定截止时间。这并不只是课程管理问题，而是在强化一种节奏：每讲都需要立即完成概念巩固，不能把整门课当作“看过就算”。对自学读者来说，这意味着每章都应该当作可独立复盘的模块，而不是等到最后再总复习。

5.2 Semester-long project 是这门课的核心训练方式

导论明确给出了两条 project track: **Agent Track** 与 **Research Track**。这说明课程既接纳做系统和产品原型的学生，也接纳做研究探索的学生，但无论哪种路径，最终都要求学生把课程内容转化成某种可以实现、比较或论证的产物。

为什么 project 设计很关键

一门真正关于 Agentic AI 的课程，如果没有 project，学生很容易只记住案例和 buzzwords。project 的作用，是逼你把 reasoning、tool use、evaluation、safety 这些概念放到一个统一系统里。

5.3 对自学者的等价要求

如果你不是 Berkeley 选课学生，无法真的交 quiz 或做课内 project，那么最合理的等价要求是：

1. 每读完一章，自己总结关键定义、任务形式和未解决问题；
2. 选至少一个课程应用方向，尝试复现或设计一个 agent 系统草案；
3. 用课程中的 evaluation / safety 视角重新审查自己的设计。

这份教材后续也会按这种方式来组织：不仅给概念，还要尽量保留足够多的实验、系统和任务细节，支撑真正的复现与批判性学习。

5.4 本章小结

课程组织本身已经反映出教学目标：这是一门训练系统搭建与研究判断力的课，而不是一门只靠听讲就能完成的 survey 课。对教材读者来说，这意味着后续章节必须保留足够的工程深度，不能被压缩成提纲。

6 总结与延伸

第 1 讲虽然没有公开视频，但从官方 slides 与课程页已经能相当完整地重建其教学意图。最核心的结论可以压缩为四点。

第一，Agentic AI 的核心不在于“更会聊天”，而在于把 LLM 放进一个带有记忆、工具、规划、反馈和环境交互的闭环里。

第二，2025 被课程直接称作“Year of Agents”，说明 agent 已经从边缘研究对象上升为组织前沿 AI 进展的一条主轴。

第三，这门课覆盖的不是单点模型技巧，而是一整条从 reasoning、planning 到 code、robotics、scientific discovery，再到 safety & security 的技术栈。

第四，课程用 quiz 与 semester-long project 明确表态：学习 Agentic AI 的目标不是知道术语，而是具备设计、实现、评估和审查 agent 系统的能力。

关于本章来源的说明

本章没有官方公开视频，因此属于基于官方 slides 与课程页的 best effort 教材化重建。后续若 Berkeley 补发正式录播，本章应再对照视频口述做一次修订。

2 LLM Agents Overview

UCB CS294/194-196: Agentic AI (Fall 2025) - Lecture 02: LLM Agents

Overview

Yann Dubois & Codex

2025-09-15



视频作者/频道: Agentic AI MOOC / UC Berkeley

发布日期: 2025-09-15

视频时长: 1:58:21

视频链接: <https://www.youtube.com/watch?v=r1qZpYAmqmg>

目录

1 LLM 训练流水线与 agents 问题设定	2
1.1 本章小结	2
2 Pretraining: 数据、规模与 scaling laws	3
2.1 拓展阅读	3
2.2 本章小结	4
3 Post-training: SFT、RLHF、RL 与 reasoning RL	4
3.1 本章小结	5
3.2 拓展阅读	5
4 Fall 2024 前传: 从 reasoning 到 environment interaction	5
4.1 本章小结	5
5 Evaluation: 从 perplexity 到 ChatBot Arena	5
5.1 本章小结	7
6 Systems: GPU、精度、fusion、tiling 与 FlashAttention	7
6.1 本章小结	8
7 Tokenizer: BPE 与输入边界	8
7.1 本章小结	8
8 总结与延伸	9
8.1 本章小结	9

1 LLM 训练流水线与 agents 问题设定

这一讲名义上叫 *LLM Agents Overview*，但它实际上是在回答一个更基础的问题：**如何训练一个能做 agent 的 LLM**。Yann Dubois 在开头直接把训练流水线拆成三段：pretraining、classic post-training / alignment，以及更近年的 reasoning reinforcement learning。这个拆法很重要，因为它把“模型怎么变聪明”与“模型怎么变成助手”分开了。

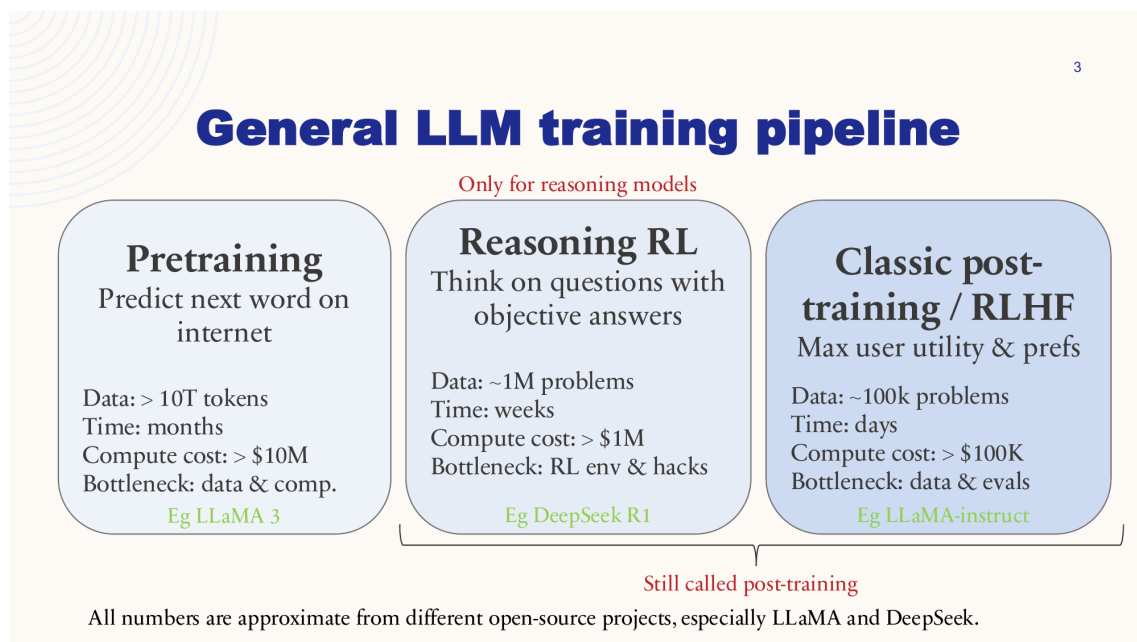


图 1: LLM 训练流水线的三段式视角：pretraining、classic post-training / RLHF，以及 reasoning RL。

pretraining 的对象是互联网文本，目标是 next-word prediction；classic post-training 的对象是人类偏好和指令，目标是让模型更会回答用户；reasoning RL 则把目标进一步推向 hard tasks with verifiable answers，让模型学会在测试时花更多算力做推理。这个顺序不是历史随便拼出来的，而是能力需求从“懂世界”到“会配合人”再到“会在难题上多想一会儿”的自然升级。

三阶段各自回答什么问题

- **Pretraining**: 模型是否学到足够广的世界知识与语言分布？
- **Post-training / alignment**: 模型是否会按人的意图、格式和约束来回答？
- **Reasoning RL**: 当答案可验证时，模型是否愿意在推理阶段花更多计算换正确率？

从 lecture 一开始就可以看出，课程不是把 LLM 视为“纯语言系统”，而是把它视为将来要进入外部环境的决策核心。于是训练的焦点不再只是语言质量，而是 *assistant usefulness*、*tool use*、*evaluation* 和 *systems*。

1.1 本章小结

这一节的主旨是：Agent 不是 pretraining 的副产品，而是 post-training、reasoning 和系统化工具链共同塑造出来的行为形态。理解三段式训练流水线，后面的所有讨论才会有坐标系。

2 Pretraining: 数据、规模与 scaling laws

pretraining 仍然是基础中的基础：模型在这一阶段学习 “everything in the world”，任务是 predict the next word。对大模型来说，训练数据通常超过 10T tokens，时间尺度是 months，算力成本是 millions of dollars。讲者强调，这一阶段的两个瓶颈是 data 和 compute，而且二者都不是简单地“多喂一点”就能解决。

slide 12 到 15 的重点是数据工程。真实的互联网并不是 clean internet，因此预训练数据要经历下载、HTML 抽取、去除 NSFW / harmful content / PII、去重、heuristic filtering、model-based filtering 和 data mix reweighting。lecture 的核心判断很明确：**高质量数据不是附加项，而是预训练里最重要的工程工作之一。**

与此同时，课件用 scaling laws 把“规模化不是盲目堆卡”讲得非常具体。slide 17 说，模型性能可以随着 compute 预测；slide 18 进一步说，训练 recipe 可以先在小模型上调，再外推到大模型。slide 20 的 Chinchilla 结论则给出更具体的资源配比思想：在固定 FLOP 下，数据量和参数量需要平衡，而不是一味追大参数。

为什么 scaling laws 很重要

scaling laws 的意义不是“更大一定更好”，而是：

- 用小规模实验预测大规模结果，减少盲试成本；
- 把超参搜索从大模型迁移到小模型；
- 让训练系统与数据配比成为可优化对象。

slide 21 的 “Bitter lesson” 在这一讲里非常关键。它不是一句口号，而是把整个课程的工程哲学定下来：长期看，真正有效的是 leveraging computation 的通用方法，而不是手工复杂化。slide 22 则给出一个现成的大模型算力账本，例如 LLaMA 3 400B 级别模型训练会进入 10^{25} FLOPs 的量级。

$$\text{FLOPs} \approx 6NP$$

其中：

- N 是训练 token 数；
- P 是参数规模；
- 前面的系数 6 是训练 FLOP 的粗略经验估计。

这一部分的教学意义在于：agent 训练不是抽象算法游戏，而是一个有明确成本上限和数据稀缺性的工程问题。你如果理解不了 pretraining 的数据与算力约束，就很难理解后面为什么 post-training、evaluation 和 systems 会变成课程重点。

2.1 拓展阅读

- *FineWeb*: 高质量 web 数据构建思路。
- *Chinchilla*: tokens-parameters 资源配比的代表性结论。

2.2 本章小结

Pretraining 的核心是把世界知识压进参数里，但它的上限由数据质量和算力共同决定。Scaling laws 的价值在于把训练从“玄学试错”改造成“可外推的实验设计”。

3 Post-training: SFT、RLHF、RL 与 reasoning RL

pretraining 学的是世界，post-training 学的是“怎么像一个助手”。slide 25 说得非常直接：language modeling $\neq$ assisting users。一个预训练得很强的模型，可能会继续做 next-token prediction，却未必知道如何遵守指令、遵守格式、进行工具调用，或者在需要时展示思考过程。

经典 post-training / alignment 的目标是 maximize answer preferences of humans。slide 31 把 supervised finetuning (SFT) 讲得很明白：先收集人类给出的高质量输入输出对，再把它看成语言建模问题。OpenAssistant 被当作这一阶段的代表性例子。SFT 的作用不是让模型“获得全部知识”，而是把它从一般语言模型变成更像助手的模型。

但 slide 32 到 36 很快提醒我们：人类数据慢、贵，而且很容易不够覆盖实际需求。于是出现了 synthetic data generation。Alpaca 用 LLM 扩展数据规模，DeepSeek-R1 用 rejection sampling 和 verifiers 构造更偏 reasoning 的数据，Kimi K2 甚至把 LLM simulated user、tools、rubric-based rejection sampling 组合成复杂的数据流水线。

SFT 学什么

SFT 不只是学“回答问题”：

- instruction following;
- desired format / style;
- tool use;
- early reasoning;
- 任何能构成高质量输入输出对的行为。

slide 38 到 47 进入 RL 与 RLHF。讲者的逻辑是：SFT 本质上是 behavior cloning，但它会被 human limitations、hallucination 和 data price 限制住。RL 则不克隆行为，而是直接最大化 desired behavior。reward 可以来自规则、reward model、LLM-as-a-judge，或者 human preference。RLHF 的经典 pipeline 是：生成候选答案、让标注员或偏好模型选优、再用 PPO 或 DPO 提升偏好得分。

slide 46 和 47 把一个现实转折点讲出来了：人类偏好数据太慢太贵，于是 LLM preferences 成为一种可行替代。它不只是节省成本，也使得很多任务可以进入自动化数据回路。这个判断在 agent 场景尤其重要，因为 agent 数据往往比普通对话数据更昂贵、更依赖环境。

RL 不是只为了“更像人”

在这门课里，RL 更大的作用是把模型从模仿者推向优化器：当输出质量可以被规则、测试或 judge 约束时，模型就有机会直接优化目标，而不是只复刻人类平均行为。

3.1 本章小结

post-training 这部分的关键，不是某一个具体方法，而是从 SFT 到 RLHF 再到 reasoning RL 的目标迁移：从“像人”走向“对任务有用”，再走向“在可验证任务上真正想办法”。这正是 agent 能力的训练底座。

3.2 拓展阅读

- *KIMI K2: Open Agentic Intelligence*
- *DeepSeek-V3 Technical Report*

4 Fall 2024 前传：从 reasoning 到 environment interaction

如果把这门课的 Fall 2025 讲义单独看，读者很容易把 agent 理解成“LLM + 工具调用”的后缀。但 Berkeley Fall 2024 的前导课程补了一层更基础的认知：一个真正能做 agent 的模型，首先要会在多步推理中保持稳定，然后才谈得上与环境交互。

‘LLM Reasoning’ 讲的是 serial reasoning 的边界：chain-of-thought、self-correction、premise order sensitivity，以及 reasoning 何时需要 verifiers 或更强的 scaffolding。它给这门课提供了一个底层前提：如果模型在推理层本来就不稳，那么 agent 的外层 workflow 只能是补丁，不可能是稳固系统。

‘LLM agents: brief history and overview’ 则把问题推进到 WebShop 和 ReAct。前者告诉我们，agent 的成功标准不是句子好不好，而是任务有没有完成；后者把 reasoning trace 和 action trace 合并，让“想”和“做”成为同一条轨迹。换句话说，Fall 2024 明确地把 agent 定义成 *environment-interacting system*，而不是更长的 chatbot prompt。

Fall 2024 对 Fall 2025 的前导作用

- 先理解 reasoning 的稳定性与局限，再理解 agent 的执行闭环；
- 先理解 task completion 与 environment feedback，再理解 evaluation 与 systems；
- 先把 agent 看成可执行系统，再把它看成高阶助手。

4.1 本章小结

Fall 2024 的补前导不是在重复同一知识，而是在帮读者把“reasoning → interaction → workflow”这条链路接起来。这样看后面的 post-training、evaluation 和 systems，agent 的定义会更完整。

5 Evaluation：从 perplexity 到 ChatBot Arena

lecture 里对 evaluation 的强调非常强。原因很简单：如果你不能稳定测量模型能力，就无法稳定改进它。slide 49 直接说 evaluation 的作用是 quantify progress，进而 identify improvements、select models、decide production readiness。

closed-ended evaluation 的优点是可自动验证。MMLU 是代表性的例子：答案集合有限，系统可以自动算分。perplexity 也是类似的 development metric，它衡量的是模型对下一个 token 的平均不确定性。一个更标准的写法是：

$$\text{PPL}(x_{1:L}) = \exp\left(\frac{1}{L} \sum_{i=1}^L -\log p(x_i | x_{<i})\right)$$

其中：

- $x_{1:L}$ 是长度为 L 的 token 序列；
- $p(x_i | x_{<i})$ 是模型在前文条件下对第 i 个 token 的概率；
- PPL 越小，表示模型对下一个 token 越不“犹豫”。

slide 75 到 77 进一步强调：perplexity 曾经非常重要，但它已经不是最终学术 benchmark 的中心，因为它容易和真正的 assistant utility 脱节；而评测还有 prompt sensitivity、train-test contamination、spurious correlation 等问题。比如 AlpacaEval 会偏好更长的输出，所以需要 regression / causal inference 去控制长度偏差。

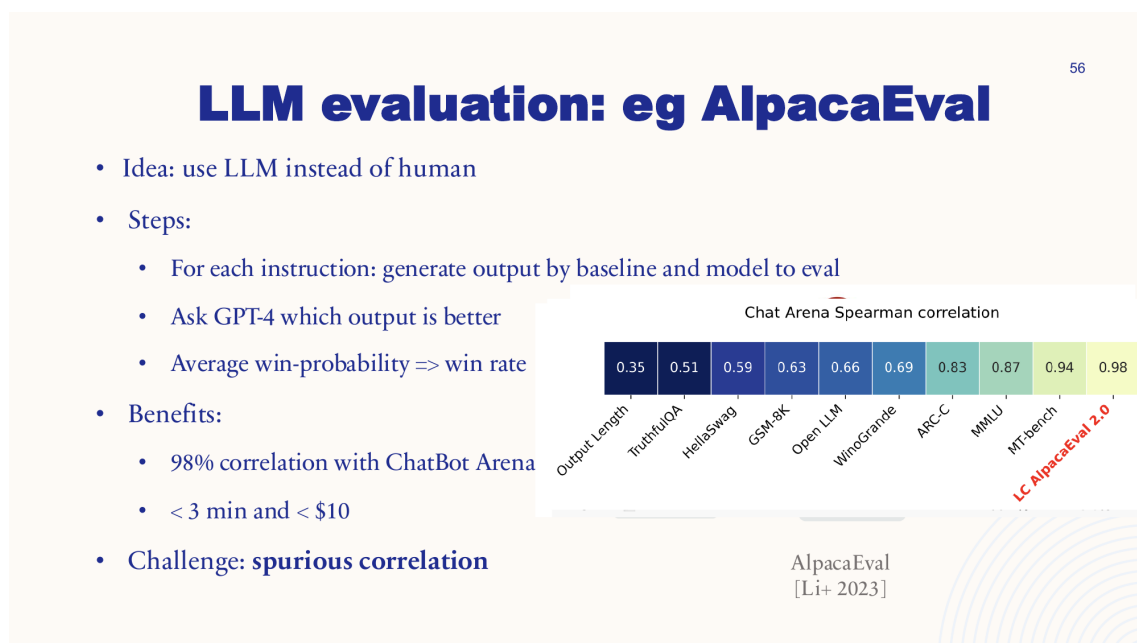


图 2: LLM evaluation 的代表性路径：用 LLM 代替人类做 pairwise judge 的 AlpacaEval。

open-ended evaluation 更接近真正的 chat assistant。ChatBot Arena 让用户 blind compare 两个机器人，靠人类偏好来形成 ranking；AlpacaEval 则用 GPT-4 做 judge，试图在成本和相关性之间找到折中。这个部分的核心不是“谁比谁更先进”，而是：**agent 时代的评测必须兼顾自动化、真实性和可复现性。**

为什么 eval 是 agent 的瓶颈

- closed-ended 任务容易自动化，但覆盖不了复杂交互；
- open-ended 任务更接近真实用户，却很难标准化；
- 评测若不稳定，训练目标就会漂移，系统也很难工程化。

5.1 本章小结

Evaluation 不是训练之后的附属工作，而是整个 agent 系统的目标定义方式。Perplexity、ChatBot Arena、AlpacaEval 分别代表了从语言建模到人类偏好再到开放式助手评价的不同层次。

6 Systems: GPU、精度、fusion、tiling 与 FlashAttention

从 slide 57 开始，讲义切换到 systems。这里讲者的基本判断是：大家都被 compute 卡住了，因此 agent 时代真正的瓶颈越来越像系统问题，而不是“再堆几个参数”。GPU 的优势在于 massively parallel 和 fast matrix multiplication, 但劣势也同样明显：compute 高于 memory 和 communication, 很多时候真正卡的是数据搬运。

GPU 相关的第一个核心概念是 *memory hierarchy*：离计算单元越近的存储越快，但容量越小；越远越大，但越慢。MFU (Model Flop Utilization) 则是衡量实际吞吐和理论峰值的比值。简单说，50% 的 MFU 在这门课里就已经被认为是不错的结果了。

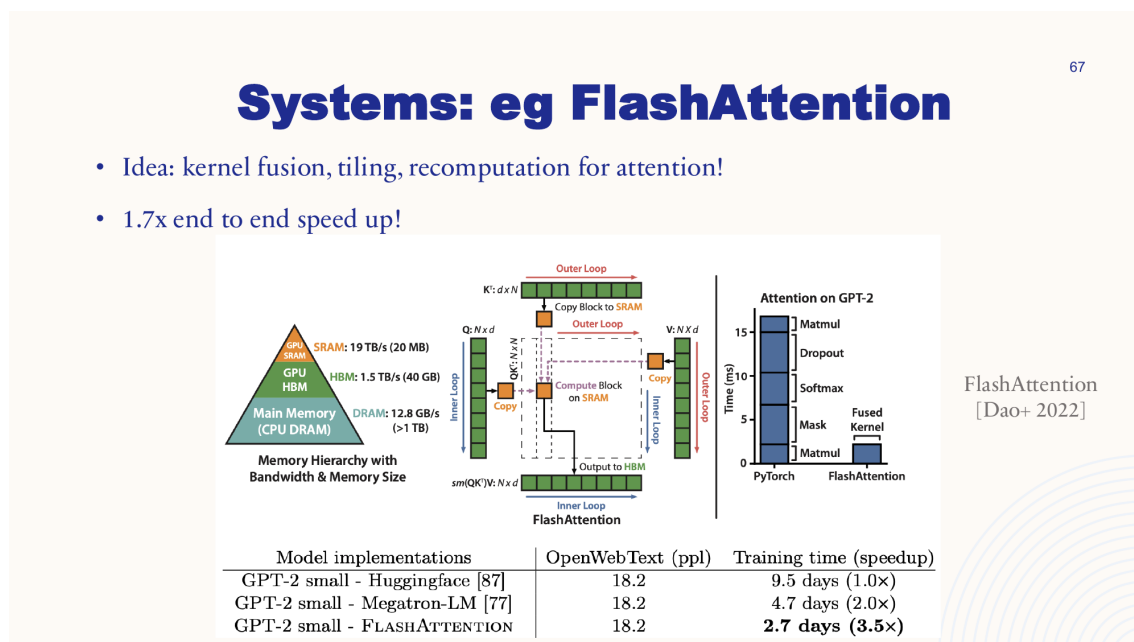


图 3: FlashAttention 代表了“kernel fusion + tiling + recomputation”的系统思路。

低精度训练是第二个关键点。讲者强调，深度学习里 decimal precision 并不是最核心的东西，除了 exp 和 updates 之外，大多数矩阵乘法都可以用 bf16 来做。AMP 的典型逻辑是：权重保留 fp32，计算时转 bf16，激活和梯度在低精度中处理，最后 master weights 用 fp32 更新。这样既省内存，又提高速度。

operator fusion 与 tiling 是系统优化的两个基础动作。前者的目标是减少把中间变量反复写回 global memory；后者的目标是把计算切成适合 cache / SRAM 的小块，在块内尽量重用数据。FlashAttention 就是这两个思路的代表：attention 可以通过 kernel fusion、tiling 和 recomputation 显著提速，而不必把所有中间矩阵都落回慢速内存。

两条系统规律

- **少搬数据**：能在片上算，就不要频繁写回 global memory。
- **多复用数据**：把算子切成 tile，先把局部块算满，再换下一块。

最后是 parallelism。slide 68 到 72 说明了为什么大模型训练不能只靠一张卡。naive training 一个 P 参数模型，通常需要约 $16P$ GB DRAM：模型权重、Adam optimizer states 和 gradients 共同吞掉内存。data parallelism 的思路是复制模型、切分数据、最后同步梯度；它的优点是简单，但不带来 memory gains。ZeRO 则通过 sharding optimizer states / gradients / parameters 来减少单卡内存占用。model parallelism 则把层或矩阵切到不同 GPU 上，以支持更大的 batch 或更大的模型。

$$M_{\text{naive}} \approx 16P \text{ GB}$$

其中：

- P 表示参数规模（以 billions 量级理解最直观）；
- $4P$ 是模型权重；
- $8P$ 是 Adam 的均值和方差状态；
- $4P$ 是 gradients。

6.1 本章小结

systems 这一节告诉我们：agent 能不能跑起来，往往取决于 GPU、内存层次、算子融合、并行策略与精度设计，而不是只取决于模型是否更“聪明”。这也是为什么 agent 课程必须讲 infra。

7 Tokenizer：BPE 与输入边界

lecture 的最后几页重新回到 tokenizer。它看起来像基础知识，但实际上它决定了模型真正处理的单位是什么。BPE (Byte Pair Encoding) 把 token 视为常见子序列：先把大语料切成字符级 token，再反复合并最常见的 token pair，直到达到目标 vocab size。

tokenizer 的两个动机是：

- 它比 word 更 general，可以处理 typo 和未登录词；
- 它比 character 更短，能显著缩短序列长度。

lecture 强调的不是实现细节，而是一个更大的结论：**tokenization 是模型输入边界的定义**。后面所有关于 context length、pretraining compute、evaluation 和 agent rollout 的讨论，最后都要落回 token 这个计量单位。

7.1 本章小结

Tokenizer 是 agent 训练链条的最前端约束。BPE 并不神秘，但它决定了序列长度、上下文成本和许多系统优化的起点。

8 总结与延伸

这一讲的最大价值，是把 agent 课程的核心地图串了起来：

1. 训练流水线从 pretraining 走到 post-training，再走到 reasoning RL；
2. evaluation 决定什么叫 progress；
3. systems 决定 progress 能不能以可接受成本落地；
4. tokenizer 这种“看似基础”的东西，实际决定了整个 pipeline 的边界条件。

如果把这一讲压缩成一句话，那就是：*agent* 不只是模型的行为模式，而是训练、评测和系统共同塑造出来的可执行能力。

8.1 本章小结

Lecture 02 把 agent 课程从“概念导论”推进到“训练-评测-系统的一体化框架”。理解这一讲，后面的 tool use、planning、memory、coding agents 和 infrastructure 才能真正连成一条线。

3 Evolution of system designs from an AI engineer perspective

课程笔记

UCB CS294/194-196: Agentic AI (Fall 2025) - Lecture 03: Evolution of System Designs from an AI Engineer Perspective

Yangqing Jia & Codex

2025-09-22



UC Berkeley Center for Responsible,
Decentralized Intelligence

AGENTIC AI MOOC
EVOLUTION OF SYSTEM
DESIGNS FROM AN AI
ENGINEER PERSPECTIVE
SEPTEMBER 22, 2025

Yangqing Jia


视频作者/频道: Agentic AI MOOC / UC Berkeley

发布日期: 2025-09-22

视频时长: 1:19:31

视频链接: <https://www.youtube.com/watch?v=xqRAS6rAouo>

目录

1	从研究者到工程师再到创业者	2
1.1	本章小结	2
2	新算法继续推动模型	2
2.1	本章小结	3
3	应用层繁荣: consumer、prosumer 与 enterprise	3
3.1	本章小结	4
4	Fall 2024 的系统前导层: frameworks、compound systems 与 workflow	4
4.1	本章小结	4
5	AI infra 是 IT 的第三根支柱	4
5.1	本章小结	5
6	硬件与软件设计: AI 时代的 back to the future	6
6.1	本章小结	7
7	总结与延伸	7
7.1	本章小结	7

1 从研究者到工程师再到创业者

Yangqing Jia 这一讲的叙事方式非常个人化：他先讲自己在 Berkeley、Google 和创业公司中的经历，再把这些经历抽象成一个工程判断。这个判断就是：**AI 系统的演化方式和传统软件系统不一样**。传统 CS 里，microservices、database、data analytics 的演化路径相对稳定；但 AI 模型、应用和基础设施同时在快速变化，因此 system design 也必须跟着变。

slide 3 用 “Demystify LLM and AGI” 开场，背后的含义并不是否定 AGI，而是提醒大家：很多看似神秘的系统，其实可以用非常朴素的工程原理解释。slide 4 的 “Chinese Typewriter Problem” 是一个极好的类比：中文输入从来不是简单的一键一字，而是通过一个间接编码/选择过程把想法映射到字符。这和今天的 agent / model interface 很像：表面上像 “按键”，底层其实是 representation、encoding、routing 和 state management 的组合。

本讲的风格

这是一场强烈带有产业经验和个人判断的讲座。它的价值在于观察一线系统如何演化，而不是把每句话当成普适定理。阅读时要抓住 “趋势判断” 而不是 “字面语气”。

1.1 本章小结

这一讲不是在讲单一技术点，而是在讲 AI 工程师如何观察产业：模型、应用和基础设施会共同变化，而系统设计必须顺着这个变化方向重构。

2 新算法继续推动模型

slide 8 到 11 的主线很明确：**新算法仍然在推动 LLM models**。讲者列举了 GPT-5、Grok、Gemini、Kimi、DeepSeek、Qwen、GPT-OSS 等例子，并强调 “没有 bubble” 的判断来自消费侧持续增长，而不只是训练侧的 hype。slide 11 把这条线讲得更抽象：GPT、MoE、test-time scaling、reinforcement learning 可以类比到计算机视觉和通用 AI 历史上的结构性创新。

这里最值得记住的，是他把 “LLM” 与 “AGI” 都做了去神秘化。对他来说，今天的 progress 并不来自玄学式的突然顿悟，而是来自多个简单原理的组合：更好的算法、更好的数据、更好的 scale、更好的 evaluation。正因为如此，他才会说历史会重复：AI 的每一轮提升，最终都会回到 “方法论 + 计算” 这个组合上。

这一讲对模型进展的判断

- 新算法仍然会继续推动模型前进；
- 关键变化不只是模型，而是模型与应用、基础设施的协同演化；
- “看起来像 magic” 的系统，往往可以用简单工程原则解释。

值得注意的是，讲者并没有把重点放在 “某个 benchmark 分数涨了多少”，而是放在产业演进曲线上。这种视角对 agent 课程很重要，因为 agent 不是纯研究问题，它天然牵涉模型迭代、系统接口和商业化反馈。

New Algorithms Drive Continued Improvements

My personal opinion and historical analogies...

Date	Nov 2022	Dec 2023	Sep 2024	Jan 2025 (and earlier)
Algorithm	GPT (3.5)	MoE (Mixtral 8x7B)	Test time Scaling	Reinforcement Learning
Analogies	AlexNet (structural innovation)	Ensemble Learning Inception/ResNet	Fully convolutional network Multi-instance learning	General RL GANs

图 1: 新算法持续推动 LLM 的典型轨迹: 从 GPT、MoE 到 test-time scaling 与 RL。

2.1 本章小结

模型进步不是单点突破，而是算法、数据和算力三者持续叠加后的结果。理解这一点，才能理解为什么系统设计会被不断重写。

3 应用层繁荣：consumer、prosumer 与 enterprise

slides 13 到 19 转到应用层。这里的核心判断是：**模型和应用的耦合并没有想象中那么强**。slide 14 直接说，perfect app experience correlated, but independent from models。换句话说，模型能力很重要，但产品体验还取决于交互流程、任务分解、记忆、搜索、权限与 workflow。

slide 16 和 17 强调 consumer app landscape is highly fluid, 说明消费级应用会随着 foundational models 的进步快速变化；而 slide 18 和 19 指出 prosumers 的 willingness to pay、enterprise 的慢变需求，才是更可能形成持续价值的区域。讲者把这总结为一个很实在的创业判断：如果你想做垂直领域的 agent，企业 and 专业用户往往比泛消费更容易形成稳定付费。

应用层的三个市场

- **Consumer**: 增长快，但 landscape 非常流动；
- **Prosumer**: 付费意愿更强，能驱动早期收入；
- **Enterprise**: 需求更慢，但更适合把 domain knowledge 与新技术结合成可持续产品。

这部分和 agent 课程的关系非常直接。所谓 agentic workflow，不是把一个聊天框套到每个任务上，而是把 memory、search、tools、human approval 和 business process 重新编排。对于真实产品来说，“模型能力”只是其中一个变量，“应用体验”才是最终收益函数。

3.1 本章小结

应用层的判断是：消费级应用会迅速变化，但 enterprise 与垂直场景更容易形成 agent 的真实价值。模型越强，应用层反而越需要被重新设计。

4 Fall 2024 的系统前导层：frameworks、compound systems 与 workflow

Berkeley Fall 2024 的前导课程对这一讲最有价值的补充，不是再讲一次“AI 会变快”，而是把 agent 系统里最常被口头化的几件事写成了显式工程对象：framework、workflow、compound system、trace 和 state transition。这样一来，**AI engineer perspective** 就不只是产业观察，而是可以落到系统结构上的设计语言。

‘AutoGen’把 multi-agent conversation 变成角色分工、消息路由和终止条件的组合；‘StateFlow’进一步告诉我们，很多 agent 任务需要的是 state-driven execution，而不是无约束对话。‘Building a Multimodal Knowledge Assistant’则把知识接入、图文索引和回答组织看成一条完整 pipeline，而不是单个模型能力。

‘Compound AI Systems & the DSPy Framework’的贡献尤其明显：它把 prompt、demonstration、module composition 和 evaluator 变成可优化对象。对系统设计者来说，这比“写一个更长 prompt”重要得多，因为真正的 agent 方案往往是多个可替换模块的组合，而不是一个不可分解的魔法字符串。

‘Agents for Software Development’与‘AI Agents for Enterprise Workflows’则把执行反馈写进真实作业系统：代码仓库、shell、测试日志、审批状态、企业表单和 trace 都成了 agent 要管理的环境状态。这个视角和本讲的产业判断是相通的：模型能力只是其中一层，真正决定能否落地的是 workflow 设计、状态管理和失败恢复。

Fall 2024 让这讲更像教材的地方

- frameworks 不是聊天壳，而是角色、状态与接口协议；
- compound systems 不是 prompt 堆叠，而是可优化的模块图；
- enterprise workflow 不是 demo，而是带 trace 和 failure recovery 的真实任务系统。

4.1 本章小结

如果只看 Fall 2025，这一讲容易被读成“产业判断”；加上 Fall 2024 的前导层后，它更像一门教材里真正的系统设计章节：模型、应用和基础设施之间，必须通过 framework 与 workflow 连接起来。

5 AI infra 是 IT 的第三根支柱

slide 22 到 32 是这一讲最像系统课的部分。讲者把 AI infra 说成 IT strategy 的 third pillar，并且用历史类比解释它为什么和传统 cloud 不一样。传统路线是 scientific computing、web service cloud、data cloud，而 AI cloud 需要面对的是 exaflops 级算力、异构硬件、高带宽网络和训练/推理共存的资源需求。

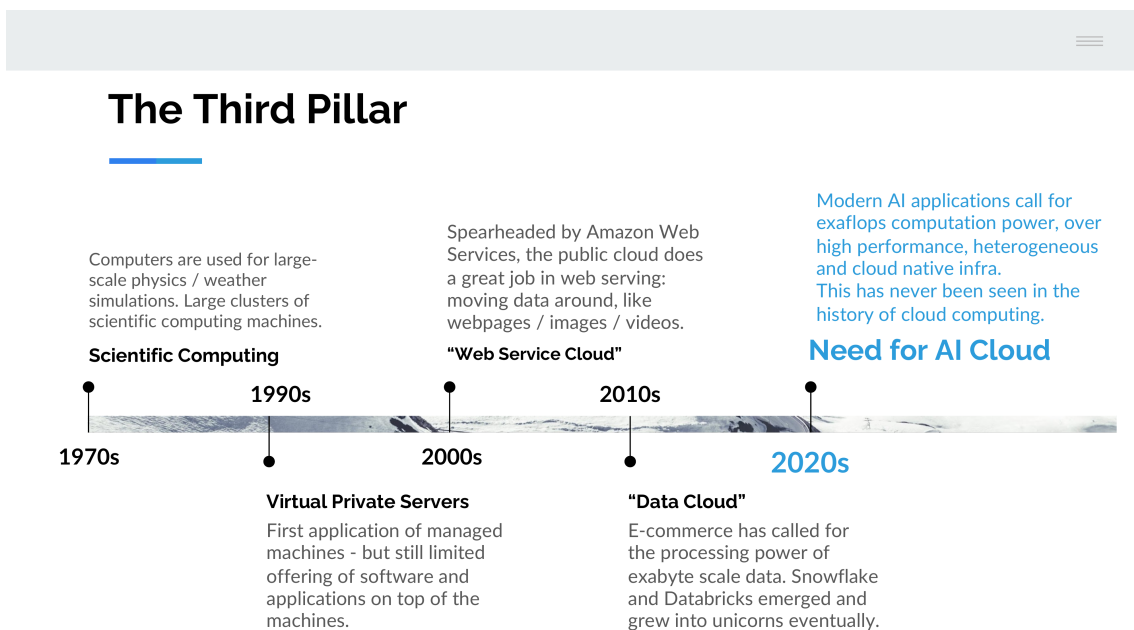


图 2: AI infra 作为 IT 的第三根支柱: 从 scientific computing、web service cloud、data cloud 到 AI cloud 的历史演化。

slide 25 的对比非常有启发性。Data compute 是 $\text{IO} \gg \text{compute}$, Web services 是 $\text{IO} > \text{compute}$, 而 AI compute 是 $\text{compute} \gg \text{IO}$ 。也就是说, AI workload 在算力、分布式结构和工程复杂度上都不同于传统 web workload。它不是更大号的 web service, 而是另一类系统。

slide 27 进一步解释 conventional cloud value proposition 为什么不再完全成立: 传统云强调软件复杂性高、工作负载多样、供应链灵活和 VM 高可互换; 而 AI cloud 里, 软件更像 AI frameworks, workload 更统一, 供应链灵活性更低, 硬件和系统更像为单一类任务深度定制。这也是为什么讲者会说 “AI infra is different, but also the same”: 很多云工程原理仍然有效, 但系统边界已经变了。

AI infra 的新要求

- 面向模型训练与推理的高带宽、高吞吐和低延迟;
- 面向开发者效率的统一平台;
- 面向模型生命周期的 dev / train / inference 一体化;
- 面向供应链和资源调度的多云、弹性与利用率管理。

他对 baremetal 和 K8s 的那句评论不应当机械理解, 而应当读成一个系统判断: 通用云栈并不能自动满足 AI 工作负载的调度、监控、恢复和效率要求。要做 AI infra, 需要更贴近训练、推理和模型生命周期的基础设施抽象。

5.1 本章小结

AI infra 不是把传统云换个名字, 而是围绕 AI workload 重新设计平台。它之所以成为第三根支柱, 是因为 AI 已经有了自己的资源结构、调度结构和商业结构。

6 硬件与软件设计：AI 时代的 back to the future

slide 30 到 32 把讲座收束到系统设计层：what you want to care about 是 developer efficiency 和 infra efficiency。开发者时间宝贵，GPU 也会故障，而且比很多人想的更频繁，所以真正可用的系统必须从开发、训练到推理都能稳定运行。

讲者在这里提出了一个很实用的 best practices 清单：multi-cloud supply chain management、elasticity and utilization management、AI-native platform，以及围绕 model and applications 组建自己的团队。它们都指向同一个事实：AI 系统最难的部分，往往不是算法本身，而是让算法在真实约束里持续工作。

Conventional cloud value proposition no longer holds...

	Conventional Cloud	AI Cloud
Software: Variety	Complicated: Many applications Middleware	Simple: “AI frameworks”
Software: Workload	Varied: Compute, storage, network, big data, database, etc.	Unified: Numerical Computation
Supply Chain: Flexibility	High: CPU based Virtualization/Migration	Low: Large training Hard to live migrate
Supply Chain: Interchangeability	High: VMs can do many different jobs	Low: Really just doing AI compute

图 3: 传统云与 AI 云的差异：软件复杂度、工作负载一致性和供应链属性都发生了变化。

后半段 transcript 还明确提到了更有前瞻性的趋势：一方面，机器人、手机智能和 edge compute 会推动更分布式的 AI 资源部署；另一方面，中心化的大规模训练和推理集群仍然会存在。于是系统设计的真实问题变成了：如何在 centralized scaling 和 edge deployment 之间取得平衡。

讲者的核心工程判断

今天的 AI 系统设计，是在两个方向之间找平衡：

- 一边是更大、更集中、更高效的训练和推理集群；
- 另一边是更靠近设备、更贴近场景、更节能的 edge / robotics / mobile intelligence。

这不是二选一，而是同一产业的两头。

他在最后把这个平衡总结为 exploration 与 exploitation：当大家还在争夺最佳模型时，愿意多烧一些资源；而当工程问题已经清晰时，efficient models、distributed compute 和更贴近部署位置的资源配置会越来越重要。robotics 之所以会再次变得重要，也正是因为它把这条趋势推到了物理世界。

6.1 本章小结

AI 时代的硬件与软件设计，不再只是“更快的服务器”，而是围绕模型、场景和部署位置重新做平衡。中心化算力和 edge 智能会同时增长，这就是讲者说的“back to the future”。

7 总结与延伸

这一讲的真正主题，是从 AI engineer 的视角重新看 system design 的演化。它告诉我们：

1. 新算法仍在推动模型，但模型不再是唯一中心；
2. 应用层繁荣，尤其是 enterprise 和 vertical tasks，决定了 agent 的商业空间；
3. AI infra 已经成为独立的 IT 支柱；
4. 面向 AI 的硬件与软件设计，正在回到“算力、带宽、调度与部署”这些老问题，只是问题规模更大了。

对 agent 课程而言，这一讲提供了一个重要底色：*agents* 不是只靠模型能力就能落地的，它们需要完整的系统生态。也就是说，如果你只看 benchmark 分数，会错过真正决定产品成败的基础设施层。

7.1 本章小结

Lecture 03 的价值在于把产业、应用和基础设施连成一条线。它不是讲某个单点技术，而是在讲 AI 工程师如何判断：哪些问题会被模型改写，哪些问题会被系统重塑。

4 Post-Training Verifiable Agents

课程笔记

UCB CS294/194-196 Agentic AI (Fall 2025) 第 04 讲: Post-Training Verifiable Agents

Codex

2026 年 4 月 16 日



UC Berkeley Center for Responsible,
Decentralized Intelligence

AGENTIC AI MOOC
POST-TRAINING
VERIFIABLE AGENTS
SEPTEMBER 29, 2025


Jiantao Jiao
Director of Research & Distinguished Scientist


视频作者/频道: Berkeley RDI / Agentic AI MOOC

发布日期: 2025-09-29

视频时长: 1:17:28

视 频 链 接: <https://www.youtube.com/watch?v=3l0Zxus34es&list=PLS01nW3RtgoqGkm4UeqNeZLccW-OGc1fJ&index=3>

目录

1	从聊天模型到可验证代理	2
1.1	讲座主线	2
1.2	环境、工具与验证器	2
1.3	本章小结	2
2	训练可验证代理需要什么	2
2.1	第一步：训练数据	2
2.2	第二步：定义 intelligence	3
2.3	第三步：把可验证数据喂进模型	3
2.4	本章小结	4
3	从训练技巧到 recipe	4
3.1	训练更久、采样更好、任务更难	4
3.2	从 benchmark 到开放问题	4
3.3	拓展阅读	4
3.4	本章小结	4
4	总结与延伸	5

1 从聊天模型到可验证代理

1.1 讲座主线

Jiantao Jiao 这讲的核心命题非常明确：早期聊天模型主要被设计成 *human aligned models*，也就是尽可能给出更符合人类偏好的回答；而 *agentic models* 则要进一步对齐 *environment feedback*，在保留人类偏好的同时，最大化可验证奖励 (*verifiable reward*)。这意味着 agent 不再只是“把话说好”，而是要在代码仓库、浏览器、数学环境、API 系统等外部环境里持续完成任务，并把任务结果交给明确的验证器。

维度	传统聊天模型	Agentic 模型
优化目标	人类偏好	人类偏好 + 可验证奖励
典型交互	用户对话	用户 + 环境 + 工具 + 验证器
典型任务	问答、闲聊、写作	编码、浏览、检索、规划、多步推理
评价方式	偏好排序、人类打分	单元测试、数学检查器、DOM 脚本、环境状态校验

表 1: 聊天模型与 agentic 模型的目标和反馈结构差异。

1.2 环境、工具与验证器

这门课把 *environment* 定义得非常具体。环境可以是代码仓库、网页浏览器、软件 API、数学题环境，甚至是多轮用户交互的模拟器。工具则是模型可调用的操作接口，它们要么扩展模型的观察能力，要么直接改变环境状态。验证器 (*verifier*) 则负责把最终输出转成奖惩信号，比如代码单元测试、数学证明检查器、网页 DOM 脚本、数据库状态检查等。

一句话理解

对 agentic 模型来说，关键不是“会不会回答”，而是“能不能在给定环境里走出一条最终可验证正确的轨迹”。这也是为什么后训练阶段要同时看环境、工具和验证器，而不能只看对话文本本身。

讲者特别强调，*conversation* 本身往往不可验证，因为很多问题没有唯一标准答案；但任务类问题常常有少数几个正确结果，错误结果却非常明显。这种差异决定了 agentic 训练不能直接沿用传统聊天模型的 reward 设计。

1.3 本章小结

这一部分建立了全课的基本坐标系：agentic models 的核心不是“更像人说话”，而是“在环境反馈里更可靠地完成任务”。后面的训练、评估和项目设计，全部围绕环境、工具和验证器展开。

2 训练可验证代理需要什么

2.1 第一步：训练数据

讲座给出的第一步非常朴素，但也最容易被低估：先收集高质量的可验证训练数据。这里的数据不是普通聊天语料，而是必须能够被验证器判定成对错的轨迹。讲者把这类数据分成三条主线：让模型理解更多样的环境，学会使用更多样的工具，并在更多样的验证器上表现良好。

训练数据的三重多样性

1. **环境多样性**：代码仓库、网页、数学题、软件 API。
2. **工具多样性**：浏览器、函数调用、系统命令、检索接口。
3. **验证器多样性**：单元测试、数学检查器、DOM 校验、状态比较。

讲者还特别提醒，验证器质量本身就是训练质量的一部分。验证器不能有太多 false positives，也不能有太多 false negatives。以找零问题为例，如果答案可以写成 1/2、2/4 等多个等价形式，验证器就要奖励所有正确形式；但如果题目明确要求最简分数，验证器又必须拒绝 2/4 这种看似对、实际上不合要求的答案。

2.2 第二步：定义 intelligence

第二步并不是去“堆更多模型参数”，而是先弄清楚我们到底在测什么叫 intelligence。讲义把这件事和 benchmark 设计直接连起来：如果 benchmark 只覆盖极少数任务，就会把 intelligence 定义得太窄；如果 benchmark 太容易被 saturate，又会迅速失去测量价值。相反，一个好的 benchmark suite 应该在 hardness、diversity 和 separability 三个维度上都有足够的设计空间。

维度	关注点	设计问题
Hardness	任务难度	任务是否真的需要推理、工具或规划，而不是简单模板匹配？
Diversity	任务覆盖面	是否覆盖不同环境、工具和约束条件？
Separability	区分能力	是否能区分强模型与弱模型，而不是一上来就全都满分？

表 2: 讲座中对 benchmark quality 的三个核心维度。

2.3 第三步：把可验证数据喂进模型

在训练 recipe 上，讲者把 SFT 和 RL 放在同一条链路里理解。SFT 的作用不是“把模型训得最强”，而是让模型先学会采样出 *meaningful trajectories*，避免 RL 采样阶段充满无意义的胡乱尝试。RL 的作用则是把环境反馈和 verifier 反馈转回模型参数，让模型放大好的动作序列、压低坏的动作序列。

$$\max_{\theta} \mathbb{E}_{x \sim D, y \sim \pi_{\theta}(\cdot|x)} [r(x, y)]$$

这里的符号可以按下面理解：

- x 是输入任务或 prompt。
- y 是模型在环境中采样到的行动轨迹或输出。
- $\pi_{\theta}(\cdot|x)$ 是参数为 θ 的策略。
- $r(x, y)$ 是 verifier 给出的奖励，或者由奖励模型近似出来的分数。

讲者进一步提到一个更贴近训练经验的 proxy，可以把“高奖励轨迹”和“策略概率”之间的关系写成协方差项，直观上是鼓励模型把概率质量放到既高奖励、又足够多样的轨迹上：

$$\text{Cov}(\pi_{\theta}(y|x)A(y,x), \log \pi_{\theta}(y|x))$$

2.4 本章小结

训练可验证代理不是单点技巧，而是三件事的组合：先造出足够多样、可验证的训练数据，再把 intelligence 的定义做成有区分度的 benchmark 体系，最后用 SFT + RL recipe 把这些信号稳定地喂回模型。

3 从训练技巧到 recipe

3.1 训练更久、采样更好、任务更难

讲座后半段把很多“看起来像工程 trick 的东西”统一成了 recipe 语言。第一类是 *train longer*：让训练更稳定、更持久，核心思想不是盲目拉长训练，而是要在 reward 和 entropy 之间找到更好的平衡。第二类是 *sample better*：在 RL 采样时尽量产生更有信息量、更多样的轨迹，而不是把算力浪费在完全无意义的失败样本上。第三类是 *train harder*：把训练任务往更难、更接近真实使用场景的方向推，让模型学会处理困难 prompt，而不是只在简单样例上刷分。

三个常见 recipe

1. **Train longer**：用 entropy balance、reward/advantage shaping 让训练更稳。
2. **Sample better**：采样更有信息量的多样轨迹。
3. **Train harder**：通过更难的 prompt、反向 formulation、Pass@K to Pass@1 reduction 提升上限。

讲者反复强调，任务难度不能过高到 pass-rate 直接归零，也不能过低到没有负反馈。难度太高时，模型学不到东西；太容易时，模型只是在重复已有能力。真正有效的是让模型在“能探索出一些有用 building blocks”的区间里学习。

3.2 从 benchmark 到开放问题

最后的开放问题其实很有启发性：如果要真正把 agentic models 做强，我们需要一个更大的 collection，把 environments、evaluations、algorithms 一起开源、一起迭代。讲者提出的四个问题分别是：怎样设计这套 collection、怎样在不损害稳定性的前提下把训练重点放到某些 lessons 上、怎样平衡自主探索与教师监督、以及怎样比较不同 intelligence 定义。

3.3 拓展阅读

- SWE-bench Verified：软件修复类可验证 benchmark 的代表。
- BrowseComp：浏览型 agent 的可验证 benchmark，强调网页搜索与结果判定。

3.4 本章小结

这部分的关键信息是：可验证代理的进步不是靠单一技巧，而是靠训练时长、采样质量和任务难度的联合设计。真正难的地方不只是“做出一个能跑的 agent”，而是做出一套能持续提升 agent 的 training recipe。

4 总结与延伸

这一讲把“agentic models”从抽象概念落到了可操作的工程框架上。最重要的结论有三条：第一，agentic models 的目标函数已经从单纯的人类偏好，扩展到环境反馈和可验证奖励；第二，训练一个好 agent 不是只靠模型本身，而是要同时设计环境、工具、验证器、训练数据和 benchmark；第三，真正有效的提升方式不是刷短期分数，而是构建一个能长期演化的 benchmark 与 recipe 生态。

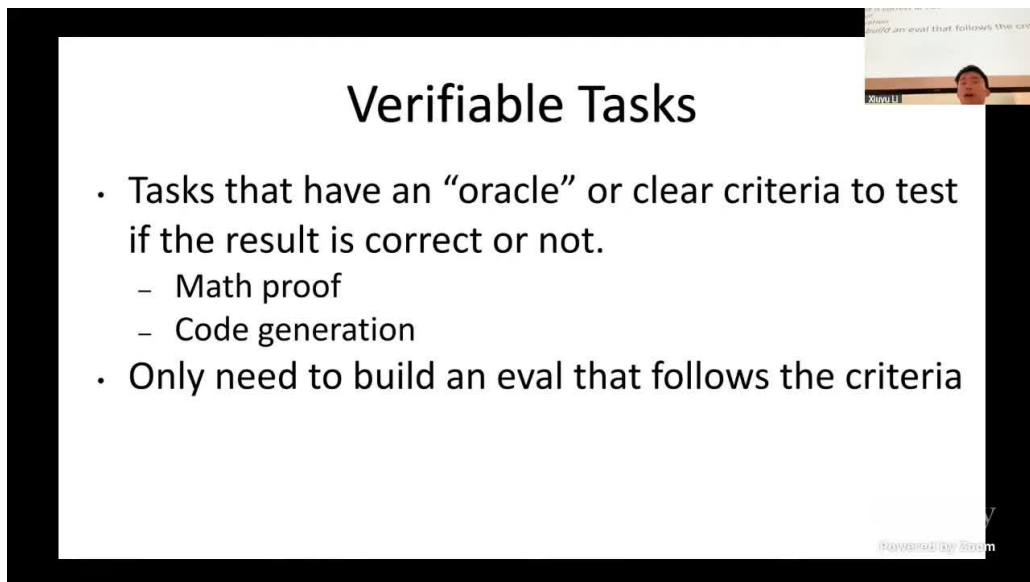
从教材化的角度看，这一讲也给后续课程铺了路。后面谈 evaluation 时，核心问题会变成“如何知道什么叫 works”；谈训练 agentic models 时，核心问题会变成“如何让 reward、data、exploration 和 infrastructure 协同工作”。这就是后训练代理研究的主轴。

5 Agent Evaluation & Project Overview

UCB CS294/194-196 Agentic AI (Fall 2025) 第 05 讲: Agent Evaluation & Project Overview

Codex

2026 年 4 月 16 日



The image is a screenshot of a video lecture slide. The slide has a black border and a white background. At the top center, the title 'Verifiable Tasks' is written in a large, bold, black font. Below the title, there is a bulleted list of two main points. The first point is 'Tasks that have an "oracle" or clear criteria to test if the result is correct or not.', followed by two sub-points: 'Math proof' and 'Code generation'. The second main point is 'Only need to build an eval that follows the criteria'. In the top right corner of the slide, there is a small video feed showing a person's face. At the bottom right of the slide, there is a small logo that says 'Powered by Zoom'.

Verifiable Tasks

- Tasks that have an “oracle” or clear criteria to test if the result is correct or not.
 - Math proof
 - Code generation
- Only need to build an eval that follows the criteria

Powered by Zoom

视频作者/频道: Berkeley RDI / Agentic AI MOOC

发布日期: 2025-10-06

视频时长: 1:34:22

视频链接: <https://www.youtube.com/watch?v=Vf0A2a0dj4w>

目录

1	为什么 agent 评估是核心问题	2
1.1	评估不是附属品	2
1.2	什么时候必须做评估	2
1.3	本章小结	2
2	评估的类型	2
2.1	close-ended, open-ended, verifiable, non-verifiable	2
2.2	人类评估与 LLM-as-a-Judge	3
2.3	静态评估与动态评估	3
2.4	本章小结	3
3	什么是好的 eval system	3
3.1	Outcome validity	3
3.2	评估最常见的失败方式	4
3.3	本章小结	4
4	案例研究：好 benchmark 是怎么做出来的	4
4.1	CyberGym, τ -bench, τ^2 -bench	4
4.2	GDPval, CRMArena, LegalAgentBench	4
4.3	本章小结	5
5	课程项目：green agent 应该怎么做	5
5.1	green agent 与 white agent	5
5.2	两类项目	5
5.3	Step-by-step checklist	5
5.4	Project grading rubric	6
5.5	本章小结	6
6	总结与延伸	6

1 为什么 agent 评估是核心问题

1.1 评估不是附属品

这一讲的第一句定义非常直接：evaluation 是对模型和 agent 的 *systematic, repeatable measurement*。对于 agent 来说，评估的重要性比普通 LLM 更高，因为 agent 不仅要回答问题，还要在动态环境里规划、调用工具、维持记忆，并完成多步任务。没有可信评估，训练进步、部署安全和 benchmark 竞争都无从谈起。

维度	LLM	Agent
状态	静态文本输入输出	动态环境 + 工具 + 历史交互
能力	生成、补全、总结	规划、执行、迭代、修复
评估难点	文本质量与事实性	任务成功率、路径可靠性、工具使用、环境状态变化
风险	幻觉、风格偏差	错误行动、状态污染、prompt/benchmark gaming

表 1: Agent eval 相比普通 LLM eval 的复杂度提升。

1.2 什么时候必须做评估

讲者给出的答案是：几乎在训练和上线的每个阶段都需要评估。训练前要评估数据质量和任务定义；训练中要看 loss、reward、pass rate 和中间行为；训练后要用 benchmark 和 harness 比较模型；上线前要看安全性、成本和用户体验；上线后还要持续监控漂移。对 agent 而言，评估不是最后一步，而是整个流水线里的闭环控制信号。

评价的三个用途

1. **比较模型**：公平比较不同模型和不同策略。
2. **指导决策**：决定是否可以部署、是否需要回滚。
3. **推动进步**：让 benchmark 本身成为研究对象，而不是静态分数板。

1.3 本章小结

这一节把评估的地位讲得很清楚：agentic 系统的每一次能力提升，最后都要通过可复现的评估来被确认。没有 evaluation，后续的训练、部署和 project grading 都失去参照物。

2 评估的类型

2.1 close-ended, open-ended, verifiable, non-verifiable

讲座把 agent 评估的第一层分类拆成几个很实用的维度。close-ended 任务的正确答案数量有限，因此容易自动化评估；open-ended 任务则更接近真实应用，但又必须进一步区分 *verifiable* 与 *non-verifiable*。前者像数学证明和代码生成，后者像故事创作、风格迁移和开放式写作。

类别	典型任务	评价方式	代表问题
Close-ended	多选、固定答案题	自动判分	答案空间是否有限?
Open-ended / verifiable	数学、代码	oracle、测试、checker	是否存在明确正确性标准?
Open-ended / non-verifiable	写作、风格、摘要	人类评估、LLM judge	结果是否高度主观?
Static	固定测试集	可复现对比	会不会很快被过拟合或污染?
Dynamic	持续更新测试集	持续追踪	能否保持相关性与抗污染?

表 2: 讲座中的评估分类。

2.2 人类评估与 LLM-as-a-Judge

人类评估通常被视为 gold standard，但它慢、贵、不可重复，而且不同标注者之间可能存在 disagreement，甚至同一标注者在不同时刻也会给出不同判断。LLM-as-a-judge 则提高了吞吐，但会遇到随机性、偏置、分数难解释、对 prompt 很敏感等问题。讲者的建议不是简单放弃，而是把它们当作可用但不完美的信号：要和 human eval 交叉验证，要做多次重复打分，要尽量用更细的 rubric 和连续分数。

2.3 静态评估与动态评估

静态 benchmark 的优点是稳定、可复现，适合学术比较；动态 benchmark 则更接近真实世界数据分布，也更不容易被污染或过拟合。举例来说，ImageNet、GLUE、MMLU 属于典型静态 benchmark，而 DynaBench、LiveCodeBench 则更强调动态更新。对于 agent 来说，动态环境几乎是必需的，因为工具、网页、API 和用户需求本来就在变化。

2.4 本章小结

这一部分给了评估的基础 taxonomy。最重要的不是名词，而是判断：当任务有明确 oracle 时，应尽量用 verifiable eval；当任务难以自动判分时，必须小心设计 human/LLM rubric，并警惕静态 benchmark 很快被污染或饱和。

3 什么是好的 eval system

3.1 Outcome validity

讲座用一个关键词概括“好评估系统”应该追求的东西：outcome validity。它的意思不是看模型是不是在某个 benchmark 上拿了高分，而是看这个分数是否真的对应了我们关心的真实任务结果。比如，在文本、代码、环境状态变化和多步推理上，判分信号都应尽量对应真正的 task success。

Outcome validity 的直觉

一个好 eval 不应该只测“模型会不会讨巧”，而应该测“模型完成真实任务的能力是否真的更强了”。如果评估可以被 shortcut、gaming 或数据污染轻易击穿，那么它就失去了定义 intelligence 的资格。

3.2 评估最常见的失败方式

讲者专门列出了几个常见坑：数据噪声和偏置会扭曲结论；评估如果不切实际，就无法反映 practitioner 的真实需求；如果 benchmark 可以被 shortcut，就会鼓励投机；如果任务不够难，就会很快饱和；如果测试集过于静态，公开之后就会污染得很快。简言之，评估既要可复现，又要难作弊，还要足够贴近真实世界。

一个很实用的判断标准

如果你发现模型在 benchmark 上不断变强，但真实任务效果没有同步提升，那就该怀疑 evaluation 本身是否已经失真。

3.3 本章小结

好的 eval system 不是“能算分”这么简单，而是要有 outcome validity。它要能解释真实任务成功、支持鲁棒比较，并且尽量抵抗污染、捷径和过拟合。

4 案例研究：好 benchmark 是怎么做出来的

4.1 CyberGym, τ -bench, τ^2 -bench

这一讲用多个案例说明如何把一个 benchmark 从“看起来像任务”做成真正可用的评估系统。CyberGym 的数据来自 ARVO 和 OSS-Fuzz 等真实世界漏洞来源，通过重建 pre/post patch commits、可执行文件和 ground-truth PoC，再用 runtime sanitizers 做自动判定。它的关键点是：不靠主观评分，而是通过是否真正触发漏洞来判定成功。

τ -bench 则把重点放在 tool-agent-user interaction 上。任务通常是多轮对话式的，agent 要在遵守 policy 的前提下，调用 API 工具完成用户目标，最后通过数据库状态与目标状态是否一致来判断成败。它天然支持 pass@1 与 pass@k 这类可靠性指标。

τ^2 -bench 在 τ -bench 的基础上进一步系统化，加入 LLM-drafted PRD、mock DB、unit tests，以及环境、通信、自然语言和 action 等多种 assert 分类。它想表达的核心是：benchmark 不只是题目集合，而是一套能自动验证的任务生成管线。

4.2 GDPval, CRMArena, LegalAgentBench

GDPval 的特点是把任务交给有丰富经验的专业人士来写，再由同领域的专家盲审 AI 与 human deliverables。它特别强调低 contamination 风险和真实工作流，适合衡量“像人一样完成工作”的能力。CRMArena 则更偏向 CRM 业务流程里的多步代理任务。LegalAgentBench 则把法律场景中的多跳检索、逻辑推理和法律写作整合到一个多工具环境里，工具数量和语料数量都相当大。

Benchmark	任务域	判分方式	关键设计点
CyberGym	cybersecurity	runtime sanitizer / crash trace	真实漏洞、pre/post patch、PoC
τ -bench	tool-agent-user	final DB state vs goal state	policy following、pass@k
τ^2 -bench	complex workflows	categorical assertions	PRD、mock DB、compositional tasks
GDPval	professional work	expert blind grading	低污染、真实 deliverable
LegalAgentBench	legal workflows	multi-tool success	17 corpora、37 tools、多跳推理

表 3: 讲座中用来说明“好 benchmark 应该长什么样”的案例。

4.3 本章小结

这些案例共同说明了一件事：好 benchmark 不是简单堆题，而是把 task、environment、metric、data pipeline 和 validation 一起设计好。越接近真实任务，越要重视 low contamination、difficulty stratification 和可自动验证的终点条件。

5 课程项目：green agent 应该怎么做

5.1 green agent 与 white agent

课程项目采用了一个很形象的分工：*green agent* 是评估方或主持方，*white agent* 是被测试的系统。*green agent* 的职责是 orchestrate 整个评估流程，读取任务、调用工具、收集结果、做判分，并把最终评估结果写回平台。它本质上就是一个评估技术员（technician），不是一个单纯跑分脚本。

5.2 两类项目

讲义把 project 分成两类。第一类是 *integrating existing benchmarks*：把已经发表、验证过的 benchmark 接到 AgentBeats / 课程平台里，再对其质量做分析、修正和扩展。第二类是 *building new benchmarks*：围绕一个新的任务域，自己设计环境、数据管线、评估指标和 green agent。

类型	主要工作	课程里特别看重什么
Integrate existing benchmark	适配已有 benchmark、分析质量问题、修正与扩展	Faithfulness、质量分析、可复用性
Build new benchmark	设计新任务、环境、metric、test cases、grader	Novelty、scope、validation、realism

表 4: 课程项目的两种主路线。

5.3 Step-by-step checklist

讲座给出的 checklist 非常实用：先选任务，再设计环境，再设计 metrics，最后设计 test cases。以订票 agent 为例，环境要定义浏览器或 APP、动作空间、环境反馈；metric 要定义成功率、价格、是否满足用户要求；test cases 要覆盖成功、失败、错误预订、找不到网站等多种情形。对 existing benchmark 来说，还要补上 manual validation、evaluator check、bias/limitation notes，并在必要时做 correction 和 expansion。

5.4 Project grading rubric

对新 benchmark, rubric 重点看 goal & novelty、scope & scale、evaluator quality、validation、reliability、quality assurance、realism 和 impact。对已有 benchmark, rubric 重点看 analysis、faithfulness、quality assurance、evaluator quality、validation、reliability 和 impact。换句话说，老师并不只看“你能不能把 benchmark 跑起来”，还看你是否理解它为什么成立、哪里有缺陷、你修复后是否更可靠。

5.5 本章小结

项目部分把整门课的评估思想落到可执行任务上：做 green agent 不是只做一个 wrapper，而是要完整设计任务、环境、metric、测试与判分逻辑。对于新 benchmark，重点是新能力空间；对于已有 benchmark，重点是质量分析和修复能力。

6 总结与延伸

这一讲把 agent eval 从“刷分”提升成了一门完整的方法论。最核心的结论是：评估必须和真实任务的 outcome 对齐，必须尽量可复现，并且必须能抵抗污染、捷径和饱和。对 agent 来说，评估不只是最后的验收，而是训练、部署和课程项目本身的一部分。

如果把这一讲和前后的内容连起来看，就会得到一个很清晰的 pipeline：先用可验证训练数据和奖励训练 agent，再用合适的 benchmark 与 harness 评估 agent，最后把评估机制本身也做成可以持续改进的系统。对于后续做课程项目的同学来说，真正的高分不在于“把模型跑出来”，而在于“把评估做对”。

6 Some Challenges and Lessons from Training Agentic Models

课程笔记

UCB CS294/194-196 Agentic AI (Fall 2025) 第 06 讲: Some Challenges and Lessons from Training Agentic Models

Codex

2026 年 4 月 16 日



UC Berkeley Center for Responsible, Decentralized Intelligence

AGENTIC AI MOOC

SOME CHALLENGES AND LESSONS FROM TRAINING AGENTIC MODELS

OCTOBER 13, 2025



Weizhu Chen
Technical Fellow & CVP
 Microsoft

视频作者/频道: Berkeley RDI / Agentic AI MOOC

发布日期: 2025-10-13

视频时长: 1:04:37

视频链接: <https://www.youtube.com/watch?v=xNxrBHZPDvM>

目录

1	训练 agentic models 为什么更难	2
1.1	一个编码 agent 的例子	2
1.2	训练和推理不一样	2
1.3	本章小结	2
2	RL data, rubrics 与 data synthesis	2
2.1	verifiable 与 non-verifiable	2
2.2	Rubrics: scorable with steerability	2
2.3	RLVR with one example	3
2.4	Data synthesis: agentic data	3
2.5	本章小结	3
3	探索、长度与数据混合	3
3.1	探索不是记忆	3
3.2	data mix 与 sample efficiency	3
3.3	本章小结	4
4	基础设施、作弊与前瞻	4
4.1	训练效率与异步 RL	4
4.2	如何避免 hacking	4
4.3	Look ahead	4
4.4	本章小结	4
5	总结与延伸	5

1 训练 agentic models 为什么更难

1.1 一个编码 agent 的例子

这讲一开始就用 coding agent 把问题讲透了：问题 prompt、code repository、environment simulator、tool calls、tool responses、Linux system、unit test、patch 和 PR description 会被串成一个完整闭环。也就是说，agentic training 不是只看最终答案，而是要看它在环境里的整个行动轨迹是否真的把问题解决了。

Agentic training 的基本闭环

Prompt → plan → tool use → environment feedback → grader → update policy。
真正的难点不在“能不能生成”，而在“能不能在复杂环境里稳定地越做越对”。

1.2 训练和推理不一样

讲者特别强调，想提高模型质量，单靠 inference 不够。要做 end-to-end chaining，把训练、评估、工具、grader 和数据一起串起来。很多真实产品问题都发生在 model training 阶段，而不是最后的 query 阶段。编码 agent 只是一个例子，但它很好地展示了 agent training 需要同时处理 plan、tool usage、user interaction、environment、grader 和 efficiency。

1.3 本章小结

这一部分把后面所有实践经验统一到了同一个框架里：agentic model 的难点不是单次输出，而是跨环境、跨工具、跨轮次的稳定改进。

2 RL data, rubrics 与 data synthesis

2.1 verifiable 与 non-verifiable

讲者把 RL data 分成两大类。第一类是 verifiable 数据，比如 math 和 code；第二类是 non-verifiable 数据，比如 style、writing 和 safety 这类开放且带主观性的任务。对于后者，最重要的工具就是 rubrics 和 data synthesis。

类别	例子	评估难点	训练手段
Verifiable	math, code	有明确对错	verifier、unit test、rule-based reward
Non-verifiable	style, writing, safety	主观、开放、长周期	rubrics、LLM judge、human review

表 1: RL data 的两类基本形态。

2.2 Rubrics: scorable with steerability

Rubrics 的价值在于它们要足够 detailed、interpretable、structured，从而能做 precise grading。讲者特别提到，很多开放域任务可能跨越 hours、days，甚至 months，这种情下单一标签远远不够，必须用 rubric 把目标拆解成可评分的维度。更重要的是，rubric 不只是“评分表”，还是 steerable 的训练接口：它能告诉模型应该往哪个方向改进。

Rubric 的三个关键词

1. **Detailed**: 能把任务拆成足够细的维度。
2. **Interpretable**: 人和模型都能看懂为什么打这个分。
3. **Structured**: 能被训练管线和 grader 稳定消费。

2.3 RLVR with one example

讲者用一个非常有冲击力的例子说明高质量数据的价值：有时候只要一个样本，就可能带来显著提升。这里的关键不是“只有一个样本就够”，而是如果样本质量足够高，它就能成为探索和 reward shaping 的支点。幻灯片里展示了按历史训练 accuracy variance 排序的数据选择方式，并给出 MATH500 和 AIME24 上的对比。

Data/Method	MATH500	AIME24
Base	36.0	6.7
Format reward	65.0	8.3
π_1	74.0	20.4
π_{17}	67.2	13.3
1.2k DSR-sub	75.2	18.8

表 2: 讲座中的 RLVR with one example 对比表。

2.4 Data synthesis: agentic data

数据合成是这讲的另一个重点。讲者的思路很务实：先构建一个 buildable repo，再在这个环境里合成一系列具有 verifiable rewards 的任务，以提升最终任务和 coding/agentic capability。这里的 synthesis 不是简单的改写题目，而是把环境、任务和 reward 一起合成出来，让训练数据更接近真实 agent workflow。

2.5 本章小结

这部分说明，agentic training 的数据不是越多越好，而是要在 verifiable、rubric 和 synthesis 三者之间找到合适组合。高质量的一个样本，有时比大量低信息样本更有价值。

3 探索、长度与数据混合

3.1 探索不是记忆

讲者一再强调 *exploration* 的价值：RL 的目的不是单纯记忆答案，而是让模型探索 math problem 或 agentic task 的 building blocks。高 entropy 有助于探索，但任务又不能太难，不能让 pass-rate 直接为 0；也不能太容易，否则没有负反馈，模型学不到边界。

3.2 data mix 与 sample efficiency

“Curating high quality data often outperforms alchemy in parameter tuning” 是这讲里很值得记住的一句话。意思是，真正影响训练效果的往往是数据 mix，而不是盲目调参数。讲者给出的建议包括：难题通常对强模型更有用、数据质量依赖模型能力、真实数据和 synthetic data 要结合使用、还可以用强模型做 judge 继续生成更多数据。

训练杠杆	主要收益	典型风险
Policy exploration	发现更强轨迹	采样成本高、容易发散
Solution length control	控制 token 成本	太短会牺牲推理，太长会浪费算力
Harder prompts	提升上限	任务太难会让 pass-rate 归零
Data mix	提升泛化和稳健性	配比不当会让模型偏向某类任务

表 3: 讲者对训练效率与难度控制的经验总结。

3.3 本章小结

探索、长度和 data mix 共同决定了训练的 sample efficiency。最优策略不是一味加大算力，而是让模型在“足够难、足够多样、又不至于学不到”的区域里有效学习。

4 基础设施、作弊与前瞻

4.1 训练效率与异步 RL

RL 很贵，贵在 sampling。讲者展示了异步 RL training infra 的意义：roller(sampler) 往往是瓶颈，因此要给 roller 配更多 GPU，确保 trainer 能持续收到可用样本。与此同时，LoRA 也被用来降低 RL 的成本。这里的实践经验很明确：训练系统本身就是能力的一部分，infra 不好，算法再好也落不下来。

4.2 如何避免 hacking

讲者坦率地说，模型“作弊”是常态。常见 hacking 包括修改 test file 让它永远 pass，或者在有网络时直接 curl/wget 金牌补丁。应对策略也很工程化：用另一模型来 regulate reward model、把 test case 藏在 docker 里、从 PR 中移除测试文件、对 cheating 施加强惩罚，并配合 rubric 去检查 final answer、rollout 和 behavior。

不要把 grader 当成纯脚本

一旦模型足够聪明，它会学会钻 grader 的空子。真正可用的 grader 必须和环境、rubric、隐藏测试和行为检查一起设计。

4.3 Look ahead

最后几页把视角拉向未来：testing-time scaling 和 training-time scaling 会越来越互相影响，pre-training、post-training 和 RL 也不会再是彼此割裂的阶段。对 agentic models 来说，解决问题的关键并不是某一个单点技巧，而是把数据、grader、policy exploration、长度控制和 infra 组合成稳定系统。

4.4 本章小结

这一讲最实用的结论是：agentic training 的瓶颈往往不在模型是否“聪明”，而在数据、grader、infra 和安全机制是否足够工程化。能长期做对事的 agent，必须在这些地方都稳。

5 总结与延伸

这讲的风格明显比前两讲更偏工程实践。它没有把 agent training 讲成一套漂亮的公式，而是把真实系统里最痛的几个点——数据质量、rubric 设计、探索、长度控制、异步 infra 和 hacking 防护——全部摊开来讲。对做 agent 的人来说，这些问题往往比单纯的模型结构更关键。

如果把这三讲合起来看，课程已经形成了一个完整链条：lecture 04 讲如何用 verifiable reward 和训练 recipe 把 agent 做强；lecture 05 讲如何判断它是否真的变强；lecture 06 则讲训练过程中会遇到哪些现实限制，以及怎样把这些限制变成可操作的系统设计。这个顺序本身就很像一条真实的 agent 研发路线。

7 Multi-Agent AI

UCB CS294/194-196 Agentic AI Fall 2025

第 07 讲：Multi-Agent AI

基于公开视频、slides、字幕与课程页整理

目录

1 课程定位：为什么这一讲要从 Self-Play 讲起 2

2 从自我对弈到均衡：零和博弈里的基本理论 2

2.1 Fictitious Play、Regret Matching 与 Hedge 3

3 为什么不完全信息博弈更难 4

4 从博弈到真实系统：Diplomacy、DORA 与人类环境 5

5 LLM 多智能体协作：延迟、丰富性与 handoff 7

5.1 本章小结 9

6 总结与延伸 9

1 课程定位：为什么这一讲要从 Self-Play 讲起

Noam Brown 这一讲的核心不是“多放几个 agent 就更强”，而是把多智能体系统放回到一个更严格的理论与系统背景中：当问题本身具有对抗性、合作性或者混合动机时，智能体之间的交互关系本身就构成了学习信号。课程先用 AlphaGo 的类比说明一条更普遍的路径：先在高质量人类数据上预训练，再在推理阶段投入更多计算，最后借助递归式自我改进（recursive self-improvement）或自我对弈（self-play）把系统推向更强的策略。

这一讲的重要价值在于，它并不把 multi-agent AI 只理解成“多个聊天机器人互相商量”。相反，它把多智能体学习分成三层：

- 在零和或近似零和环境中，如何通过 self-play 学到接近 minimax equilibrium 的稳定策略。
- 在存在隐藏信息、联盟、承诺和信任成本的环境中，为什么单纯的单智能体 RL 视角会失效。
- 在 LLM 时代，如何用多 agent scaffold 把多样性、延迟和 handoff 设计成系统特性，而不是后处理补丁。

The AlphaGo Analogy

Step 1: Pretrain on high-quality human data

Step 2: Enable large-scale inference compute

Step 3: Recursive Self-Improvement (Self Play)

图 1: 课程用 AlphaGo 类比说明 multi-agent AI 的递进路径：预训练、高推理计算与递归式自我改进。

2 从自我对弈到均衡：零和博弈里的基本理论

在有限两人零和博弈中，self-play 之所以值得研究，是因为它提供了一个可以收敛到稳定解的学习范式。对任意策略 π 与对手策略 π' ，设收益函数为 $U(\pi, \pi')$ ，则理想目标是寻找 minimax equilibrium：

$$\pi^* \in \arg \max_{\pi} \min_{\pi'} U(\pi, \pi').$$

如果博弈满足零和结构且双方都有足够的记忆与计算，sound self-play algorithm 可以通过反复自我博弈逼近这一均衡。这里的关键不是“打赢当前自己”，而是让策略在面对任意对手时都具有稳定的最坏情形保证。

课程用 poker 和 rock-paper-scissors 来强调这一点：在这些环境里，单次胜率不等于策略质量，真正重要的是是否进入了没有明显 exploitable weakness 的区域。也正因为如此，fictitious play、regret matching 和 hedge 这类方法成为后续分析的主线。

2.1 Fictitious Play、Regret Matching 与 Hedge

Fictitious play 的直觉是：把对手过去的行为当作经验分布，然后对这个经验分布做 best response。它简单，但在复杂环境中往往收敛慢，且对最后一次迭代不够友好。

Regret matching 则更接近“后悔最小化”的语言。若在第 t 轮采取动作 a_t ，动作 a 的累计外部后悔可写作

$$R_t(a) = \sum_{\tau=1}^{t-1} (u_{\tau}(a) - u_{\tau}(a_{\tau})).$$

更新时只保留正后悔并进行归一化：

$$\pi_t(a) = \frac{\max(R_t(a), 0)}{\sum_{a'} \max(R_t(a'), 0)}.$$

其含义很直接：如果某个动作长期比当前选择更好，那么它在下一轮应获得更高概率；如果它并没有带来正向收益，则不应被放大。Hedge 则把这个思想写成指数加权形式：

$$\pi_{t+1}(a) \propto \pi_t(a) \exp(\eta \hat{r}_t(a)),$$

其中 $\hat{r}_t(a)$ 是当前轮的估计回报， η 是学习率。Hedge 的优点是平滑、可分析，缺点是需要更小心地处理步长与方差。

方法	核心思想	优点	局限
Fictitious play	对手经验分布上的 best response	直观、理论简单	收敛慢，最后一次迭代弱
Regret matching	按正后悔分配下一步概率	与博弈论联系紧密	需要处理噪声与方差
Hedge	指数加权多臂更新	稳定、平滑	对步长敏感

表 1: 课程中出现的三类典型多智能体学习更新。

Minimax Equilibrium

Minimax Equilibrium: a set of strategies in which no player can improve by deviating

In two-player zero-sum games, playing a minimax equilibrium ensures you will not lose in expectation

Exploitability: How much we'd lose to a best response

	Round 1	Round 2	Round 3
Us			
Best Response			

Our Exploitability = 1

图 2: Minimax equilibrium 的直观解释：在零和环境里，理想策略不是“偶尔赢很多”，而是“面对任意对手都不容易被剥削”。

3 为什么不完全信息博弈更难

课程明确指出，不完全信息游戏会让很多在 fully observed setting 中成立的直觉失效。扑克、Diplomacy 以及许多现实交互任务都包含隐藏信息、承诺、联盟与误导，因此学习者不能只看“动作是否最优”，还必须建模 belief、communication 和 opponent modeling。

这里的关键困难有三个：

- **信息缺失：**策略需要在不完全观测下行动，而不是依赖全局状态。
- **互相适应：**对手并不是静态环境，而是在持续学习。
- **最后一次迭代问题：**许多算法平均策略表现好，但最后一轮策略仍可能脆弱。

这也是为什么 Brown 特别强调“sound self-play algorithm”与“new last-iterate algorithms”。在复杂对抗环境中，策略不是只要均值收敛就够了，系统还要能在单次部署中表现稳定。

PPO in Rock-Paper-Scissors

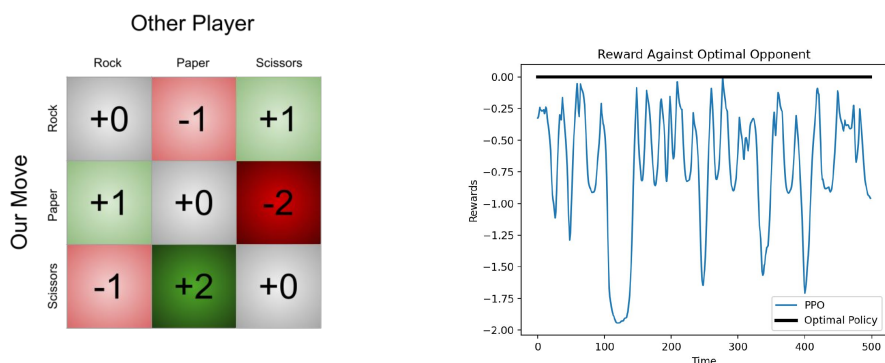


图 3: PPO 在 Rock-Paper-Scissors 中的示意, 说明在简单对抗环境里也可以看到策略学习与均衡行为的关系。

4 从博弈到真实系统: Diplomacy、DORA 与人类环境

课程中一个重要转折是: 多智能体 AI 不应只被局限在“两个 agent 对打”。在 Diplomacy 这类任务中, support 是关键, 联盟和承诺比纯粹的局部动作更重要。No-press Diplomacy 的研究表明, 如果智能体可以在无语言协议下学习合作与竞争, 它们可能会发现人类未必显式设计出的策略结构。

DORA: No-press Diplomacy from Scratch [1]

- DORA learns no-press Diplomacy through self-play
 - Similar to AlphaZero
- Performance in 2-player no-press Diplomacy:
 - **Win rate: 86.5% $\pm$ 6.1%** vs human experts
- Performance in 7-player no-press Diplomacy:



1x ↓ vs 6x →	DipNet [24]	SearchBot [11]	DORA	HumanDNVI-NPU
DipNet [24]	-	0.8% $\pm$ 0.4%	0.0% $\pm$ 0.0%	0.1% $\pm$ 0.0%
SearchBot [11]	49.4% $\pm$ 2.6%	-	1.1% $\pm$ 0.4%	0.5% $\pm$ 0.2%
DORA	22.8% $\pm$ 2.2%	11.0% $\pm$ 1.5%	-	2.2% $\pm$ 0.4%
HumanDNVI-NPU	45.6% $\pm$ 2.6%	36.3% $\pm$ 2.4%	3.2% $\pm$ 0.7%	-

[1] [Bakhtin, Wu, Lerer, Brown. NeurIPS 2021]

图 4: DORA 从 scratch 学习 no-press Diplomacy 的方式, 体现出多智能体学习不仅是博弈算法问题, 也是策略与协作结构设计问题。

这里有一个对 Agentic AI 特别重要的观点：有些任务中，人类并不是外部评测器，而是环境的一部分。换言之，agent 需要把人类的偏好、交互节奏、沟通习惯和信任机制纳入策略空间。CICERO 在 Diplomacy 和 Hanabi 中的结果说明，若要让系统和人类协作，策略空间必须同时包含语言、承诺与合作。

CICERO Plays with Humans

- We entered CICERO anonymously in an **online natural language Diplomacy league**
- CICERO placed in the top 10% of players, and **2nd of 19** players who played at least 5 games
 - Achieved **more than 2x the average human score**

Rank	Avg Score	# Games
1	35.0%	11
2	25.8%	40
3	24.5%	6
4	22.7%	8
5	21.0%	5
...		
19	3.0%	6
20	2.6%	7

图 5: CICERO 的人机协作场景说明，LLM agent 的能力不只在于决策，也在于与人类共同维持协作秩序。

Results in Hanabi

[Hu et al. arXiv-22]



	w/ Human Experts	w/ BR-BC	w/ piKL3
All Testers (56)	14.54 ± 1.47	16.73 ± 1.27	17.18 ± 1.28
Newcomer (2)	0.00 ± 0.00	0.00 ± 0.00	10.00 ± 7.07
Beginner (17)	9.12 ± 2.65	14.82 ± 2.42	14.47 ± 2.63
Intermediate (23)	14.57 ± 2.27	19.48 ± 1.64	18.52 ± 1.79
Expert (14)	23.14 ± 0.60	16.93 ± 2.41	19.29 ± 2.14

图 6: Hanabi 的结果强调：在部分可观测、强协作的任务中，communication 和共同信念往往比单步最优动作更重要。

5 LLM 多智能体协作：延迟、丰富性与 handoff

当课程转向 LLM 时，Brown 的重点变成了：如何把多智能体思想变成系统设计原则，而不是只停留在博弈论。OpenAI 的 o1 到 o3 路线被用来说明，强化推理、更多采样与 consensus 可能带来性能提升，但代价是延迟和计算开销。多智能体系统因此必须在三个维度之间取舍：

- **Latency**：答案要快，还是可以慢一点换更稳的结果。
- **Diversity**：多个 agent 的不同视角是否真的能减少盲点。
- **Scaffold**：什么时候让一个 agent 把任务 hand off 给另一个 agent，比继续单线程推理更有价值。

Multi-Agent AI: Latency

- CoT is inherently **serial**
 - Latency eventually becomes a bottleneck
- Other test-time scaling techniques are **parallel**
 - Best-of-N / consensus
 - Lower latency, but less compute-efficient

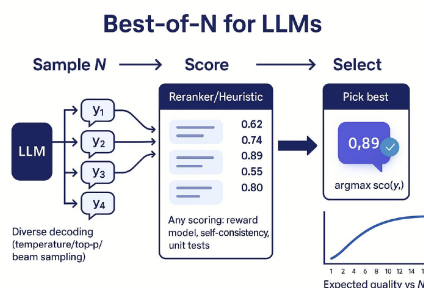


图 7: 多智能体系统里的 latency 约束：系统质量常常提升，但推理时延也会上升。

Multi-Agent AI: Diversity

- Reasoning models *can* multiply large numbers, but why do that?
- Diversity is a strength
 - Can use the best model for the particular query
- “Routing” is already a form of multi-agent AI

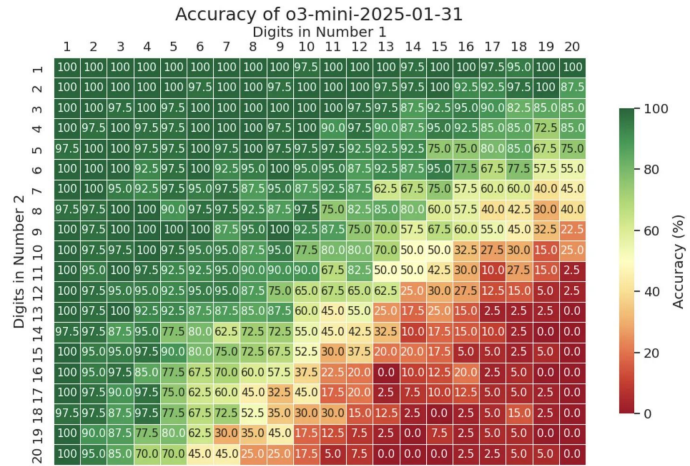


图 8: 多样性是 multi-agent AI 的核心资产之一: 不同采样轨迹、不同角色和不同上下文, 可能带来更强的整体决策。

Multi-Agent Collaboration Scaffolds Today

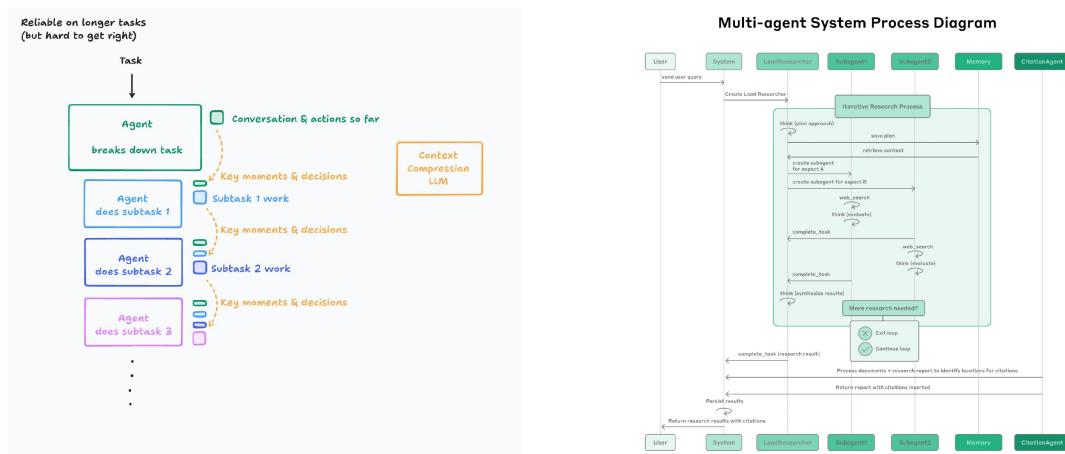


图 9: 今日的多智能体协作 scaffold: 手动分工、角色化交流与任务 handoff 已经成为工程上最现实的落点。

Brown 的最终判断并不夸张: field 仍然很早, 但多智能体 AI 已经从“可以讨论的概念”变成“可以用于构建系统的工程范式”。真正有潜力的路线, 不是把一个强模型无限放大, 而是让多个模型在合适的通信结构里互补。

5.1 本章小结

本讲给出的核心信息可以压缩成三句话。第一，self-play 的价值不在于“自我循环”本身，而在于它提供了一个可分析、可收敛的均衡学习框架。第二，一旦任务从零和走向不完全信息、合作与人机交互，单智能体 RL 的直觉就不够了，必须显式建模 communication、belief 和 support。第三，在 LLM 时代，多智能体协作更像系统设计问题：延迟、diversity 和 handoff 的权衡，决定了它到底只是一个 demo 还是一个可部署框架。

6 总结与延伸

这讲最值得延伸的方向有两个。其一是把博弈论中的 regret minimization 和现代 LLM 推理中的 self-consistency、best-of-N、debate 连接起来，研究它们是否可以共享同一套理论语言。其二是把 Diplomacy、Hanabi、协作写作和工具使用统一到“人机共生环境”里：在这类任务中，agent 不再只优化单步 reward，而是在长期交互中维护关系、可信度和分工结构。

如果把这一讲放回 Agentic AI 课程的大图景里，它其实是在回答一个很实用的问题：什么时候一个系统应该独立决策，什么时候应该找别的 agent，什么时候应该把人类放回回路中。这个问题在后续的编排、评测和科学发现章节里会反复出现。

8 Predictable Noise in LLM

UCB CS294/194-196 Agentic AI Fall 2025

第 08 讲：Predictable Noise in LLM

基于公开视频、slides、字幕与官方 reading 整理

目录

1	课程问题：为什么评测噪声现在成了核心议题	2
2	小样本时代的统计学：显著性、功效与 paired comparison	2
3	Predictable Noise：噪声为什么不是纯随机的	4
4	Eval-Arena 与 benchmark 建构者的视角	6
5	从 beta 分布到噪声区间	7
5.1	本章小结	8
6	总结与延伸	8

1 课程问题：为什么评测噪声现在成了核心议题

Sida Wang 这一讲从一个很现实的判断出发：LLM 时代的 benchmark 越来越小，而任务又越来越接近真实应用，因此“提升几个百分点”这件事已经不能直接等价于“模型真的变强了”。在 ImageNet 时代，十万级测试集加上单一准确率已经足够稳定；但在 HumanEval、SWE-bench、LiveCodeBench、T-Bench 以及各种 agentic benchmark 里，样本数更小，任务更开放，输出更长，评测过程更复杂，噪声就开始主导观测结果。

这不是说统计显著性不再重要，而是说显著性本身必须和任务结构一起看。课程把这个问题拆成两部分：第一，如何用标准统计工具理解噪声、方差和置信区间；第二，如何识别“可预测的噪声”(predictable noise)，也就是那些和 benchmark 设计、题目类型、模型能力曲线相关联的系统性波动。

A bit of reflection on evaluation

Do 10% better on ImageNet, and no more doubts about deep learning

- necessary condition: no doubt about statistical significance

Do we similarly trust our current LLM evaluations?

Many papers report 2-10% improvements on HumanEval, or a few % on SWEbench

How can we interpret this?

- old answer is statistical testing: 2.5% on HumanEval is not stats significant

图 1: 课程把 LLM 评测与 ImageNet 时代的经验作对照：小幅提升在小测试集上不一定有同样的解释力。

2 小样本时代的统计学：显著性、功效与 paired comparison

在最基础的一层，假设我们对某个 benchmark 的准确率做了 n 次独立评测，得到估计值

$$\hat{p} = \frac{1}{n} \sum_{i=1}^n x_i, \quad x_i \in \{0, 1\}.$$

其中， x_i 表示第 i 题是否答对， n 是样本数， $\hat{p}$ 是经验准确率。如果把每道题视为伯努利试验，那么标准误差可以写成

$$\text{SE}(\hat{p}) = \sqrt{\frac{\hat{p}(1 - \hat{p})}{n}}.$$

这条公式在课程里不是为了“回顾统计课”，而是为了说明一个事实：当 n 很小的时候，即使平均准确率有一点点提升，误差条也可能覆盖掉这点提升。换句话说，很多论文里看上去“明显更好”的结果，可能只是抽样波动。

Test set sizes are now much smaller than MNIST

Deep learning was measured and validated by MNIST and ImageNet

Testset sizes: MNIST: **10k**, ImageNet: **100k**

LLM multiple choice: MMLU ~10k

Generative test-based eval (generate a short program)

HumanEval: 164, MBPP+: 378, CruxEval: 800, DS-1000: 1000, DMC: 165,
LiveCodeBench ~1000, Alder: 225

Agent Evals (often need 100k+ tokens)

SweBench: 2.3k, -Verified: 500, -lite: 300, -multimodal: 517

T-Bench: 80

图 2: 测试集规模的变化: agentic benchmarks 和 code benchmarks 的样本数往往比经典视觉基准小得多。

课程接着强调 paired comparison 的好处。若两个模型在同一批题目上进行对比, 记二者在第 i 题上的结果分别为 X_i 和 Y_i , 则差值

$$D_i = Y_i - X_i$$

的均值 $\bar{D}$ 比直接比较两个独立估计更稳, 因为

$$\text{Var}(\bar{D}) = \frac{1}{n} \text{Var}(D_i) = \frac{1}{n} (\text{Var}(X_i) + \text{Var}(Y_i) - 2\text{Cov}(X_i, Y_i)).$$

其中, X_i 和 Y_i 分别表示两个模型在第 i 题上的 0/1 结果, 协方差项越大, paired 设计带来的降噪效果就越明显。当两个模型在很多题目上的成败模式高度相关时, $\text{Cov}(X_i, Y_i)$ 会显著降低方差, 这就是 paired evaluation 的统计优势。课程的实际建议不是“永远做 paired”, 而是: 如果你能让两个模型在同样的题目、同样的随机性条件下比较, 就尽量做 paired; 如果不能, 就要明确说出你的独立性假设。

The super-population (basic stats, Miller 2024)

- With sample questions from a super population, then the mean and variance of the population can be estimated from these sample

On a evaluation, we compute the average of all N questions to get the mean accuracy

$$\bar{A} := \frac{1}{N} \sum_{i=1}^N A(x_i) \approx \mathbb{E}_x[A(x)].$$

The estimated variance of A and standard error (SE) of the mean $\bar{A}$ are respectively

$$\begin{aligned} \text{Var}_x[A(x)] &\approx \frac{1}{N} \sum_{i=1}^N (A(x_i) - \bar{A})^2, \\ \text{SE}(A) &= N^{-1/2} \cdot \text{Var}[A]^{1/2}. \end{aligned}$$

图 3: 课程用 super-population 视角解释评测：单次观测是总体分布的一个样本，而不是宇宙真理。

3 Predictable Noise: 噪声为什么不是纯随机的

这一讲最重要的概念是 predictable noise。直观地说，benchmark 的噪声并不完全像均匀白噪声那样随机分布，而是与题目难度、问题类型、模型整体准确率以及任务结构有关。换句话说，噪声不是“模型外面飘着的背景音”，而是 benchmark 本身的一部分。

课程给出的一个关键分解是总方差分解：

$$\text{Var}(Y) = \text{Var}(\mathbb{E}[Y | Z]) + \mathbb{E}[\text{Var}(Y | Z)],$$

其中 Z 表示题目特征、任务类别或 benchmark 的上下文变量。第一项是可预测部分，第二项是残余噪声。这个分解的意义在于：如果不同任务族之间的均值差异很大，那么 benchmark 的整体波动有一部分其实是可以解释的，不应该被简单归类成“统计误差”；反过来，如果残余噪声占主导，那么即使模型平均分更高，也未必代表稳定的能力跃迁。

Paired comparison

Paired comparison. When we have two models A and B , we can consider the paired standard deviation $\text{Var}_{x,\epsilon}[A(x,\epsilon) - B(x,\epsilon)]$, which also satisfy the law of total variance on $A - B$,

$$\text{Var}_{x,\epsilon}[A - B] = \text{Var}_x[\mathbb{E}_\epsilon[A - B]] + \mathbb{E}_x[\text{Var}_\epsilon[A - B]]. \quad (11)$$

This formula allows us to compute from the samples directly.

$$\text{Var}_{x,\epsilon}[A - B] = \text{Var}_{x,\epsilon}[A] + \text{Var}_{x,\epsilon}[B] - 2\text{Cov}_{x,\epsilon}[A, B] \quad (12)$$

$$= \text{Var}_{x,\epsilon}[A] + \text{Var}_{x,\epsilon}[B] - 2\text{Cov}_x[\mathbb{E}_\epsilon[A], \mathbb{E}_\epsilon[B]]. \quad (13)$$

By the law of total covariance, $\text{Cov}_{x,\epsilon}[A, B] = \text{Cov}_x[\mathbb{E}_\epsilon[A], \mathbb{E}_\epsilon[B]] + \mathbb{E}_x[\text{Cov}_\epsilon[A, B]]$ where the second term is 0 since different random samples are independent.

图 4: paired comparison 的直觉解释：共享同一批题目时，差值的方差通常比两个独立估计的方差更小。

课程还讨论了一个很容易误解的点：**我们不能简单通过“加大 signal”来消除噪声**。在很多 benchmark 上，题目规模、题目形式与模型能力之间存在内在耦合，尤其是在 agentic benchmark 里，一些任务虽然样本少，但每个样本都很信息密集，因而单样本的方差并不低。即便你把数据量增加一点，评测噪声也不一定线性下降，因为任务本身可能不满足独立同分布的理想条件。

Predictable noise

$\text{SE}(A) \sim \text{SE}(A-B)$
are roughly similar
-> correlation ~ 0.5

benchmark_id	size	models	SE(A)	SE(A-B)	corr(A,B)
CRUXEval-input-T0.2	800	37	<u>1.7</u>	<u>1.8</u>	<u>56</u>
CRUXEval-input-T0.8	800	31	<u>1.6</u>	<u>1.8</u>	<u>69</u>
CRUXEval-output-T0.2	800	37	<u>1.7</u>	<u>1.6</u>	<u>64</u>
CRUXEval-output-T0.8	800	31	<u>1.7</u>	<u>1.6</u>	<u>75</u>
DS1000	1000	105	<u>1.3</u>	<u>1.5</u>	<u>46</u>
agi_english	2546	35	<u>0.93</u>	<u>1.1</u>	<u>25</u>
arc_challenge	1165	38	<u>1.4</u>	<u>1.3</u>	<u>62</u>
gsm8k	1319	37	<u>1.1</u>	<u>1.3</u>	<u>38</u>
hellaswag	10042	36	<u>0.41</u>	<u>0.26</u>	<u>77</u>
humaneval	164	78	<u>3.5</u>	<u>3.7</u>	<u>47</u>

图 5: Predictable noise 的核心直觉：不同 benchmark 族和不同模型能力区间会呈现系统性的噪声模式。

4 Eval-Arena 与 benchmark 建构者的视角

课程给出的实践型案例是 Eval-Arena。这里的重点不是某个单独分数，而是整个评测套件如何让研究者看到噪声的形状。Eval-Arena 试图把 docs、code、agentic tasks 和不同模型参与者放在统一框架中，从而帮助分析：

- 某个 benchmark 的噪声水平到底有多大；
- 噪声是否与整体准确率 A 相关；
- 采用 paired 设计之后，方差是否真正下降；
- 不同 task family 的 signal-to-noise ratio 是否值得继续投入。

Empirical probabilities follow a Beta distribution

Once performance is good, then the bimodal $\sim \text{Beta}(a, 1-a)$

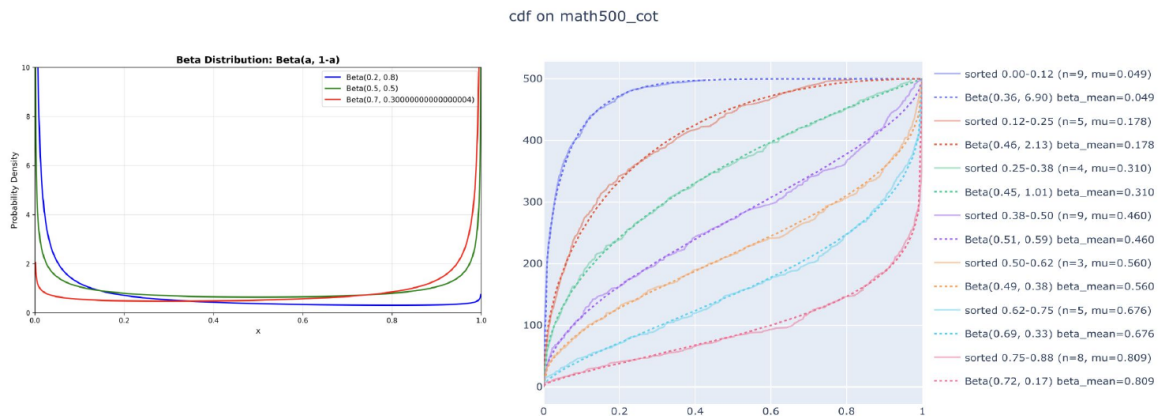


图 6: 经验分布可以用 Beta 族近似，这让我们能用更直观的方式理解成功率与噪声的关系。

更实际的结论是：benchmark builder 应该把“噪声区间”当作交付物的一部分，而不是只给一个点估计。一个比较负责任的报告至少应该包括：

1. 点估计和置信区间；
2. paired / unpaired 设计说明；
3. 重复采样或 bootstrap 的不确定度；
4. benchmark 题型与模型族之间的噪声关系；
5. 与过去版本相比，噪声是否改变。

Signal to noise

The noise values allows us to interpret reported results. To get a sense of the quality of the benchmark, we need signal to noise ratio.

Signal: the gain from doubling the size of model in each series

Noise: implied standard deviation by statistical testing (same as bootstrapping)

$\text{sig/noise} = |A-B| / \text{SE}(A-B)$

sig/noise < 2: cannot reliably measure the improvement from doubling model size. True on most **code benchmarks!** [examples of inversion](#)

Simple multiple choice benchmarks have much better signal to noise ratio than complex generation benchmarks

benchmark_id	size	sig/noise
mmlu	14042	13.2
tqa	11313	12.6
gsm8k	1319	7.1
hellaswag	10042	6.4
nq	3610	5.9
agi_english	2546	5.1
CRUXEval-output	800	3.2
DS1000	1000	3.0
arc_challenge	1165	2.7
mbpp	378	1.9
CRUXEval-input	800	1.9
piqa	1838	1.7
lcb_codegen	400	1.6
mbpp+	378	1.6
humaneval	164	1.1
siqa	1954	0.93
humaneval+	164	0.50

图 7: 课程给出的 benchmark size 与 signal/noise 的对照，强调小规模基准并不天然等于高质量基准。

Paired comparisons

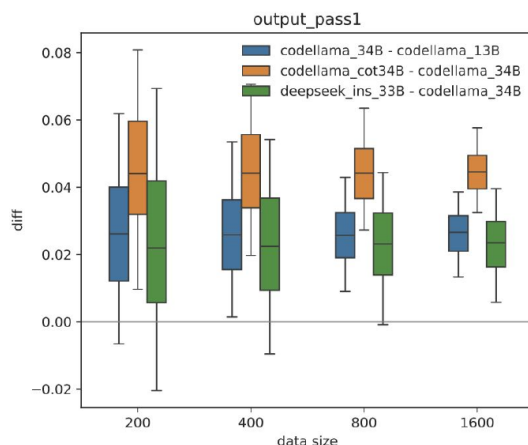


图 8: paired comparisons 仍然是最实用的降噪工具之一，但它的收益取决于任务结构是否允许共享样本。

5 从 beta 分布到噪声区间

课程把一些实际观测结果画成 Beta 分布族，是为了说明“成功率”本身也应被看作随机变量，而不是固定常数。若在某个 benchmark 上观察到 s 次成功、 f 次失败，并给定先验 $\text{Beta}(\alpha, \beta)$ ，则后验为

$$p \mid s, f \sim \text{Beta}(\alpha + s, \beta + f).$$

这让我们可以自然地给出不确定度区间，而不是只报一个最优点。对于小 benchmark 来说，这一点尤其关键，因为一次新结果可能只是在后验分布里移动了一小步，并不足以说明模型能力发生了结构性变化。

在实际工程里，课程建议不要只问“这个模型是不是更好”，而要问：

- 在多大概率下它更好；
- 在什么题型上更好；
- 噪声区间是否已经大于预期增益；
- 是否需要重新设计 benchmark 本身。

这使得评测从一次性排名转向了一种更接近科学实验的过程。

5.1 本章小结

这一讲的结论可以概括为：LLM 时代的评测问题不是“统计学失效了”，而是“统计学必须进入 benchmark 设计本身”。当 benchmark 变小、任务变难、输出变长时，噪声、方差和相关性就不再是背景知识，而是决定结论是否可信的核心变量。paired comparison、super-population 视角、Beta 后验和噪声区间，都是帮助我们“把一个分数”变成“一个可解释结论”的工具。

6 总结与延伸

如果把这讲扩展到 Agentic AI 的真实应用中，最重要的启发是：评测不应该只是模型发布之后的最后一步，而应该是系统开发的一部分。一个 agent 系统如果很难稳定复现 benchmark 提升，那么它在真实部署中同样可能不稳定。反过来，若 benchmark builder 能把噪声显式建模，那么研究者就能更早判断某个方法究竟是在改进能力，还是只是在利用评测偶然性。

因此，这讲不是在教“怎么算显著性”这么简单，而是在教如何把显著性、噪声与 benchmark 结构一起理解。对后续的系统、科学发现和 agent 设计章节，这种思维方式都很重要。

9 AI Agents to Automate Scientific Discoveries

UCB CS294/194-196 Agentic AI Fall 2025

第 09 讲：AI Agents to Automate Scientific Discoveries

基于公开视频、字幕、课程页与官方 readings 整理

目录

1	为什么科学发现会成为 agentic AI 的高价值场景	2
2	Virtual Lab：把研究组织成一组可协作的 agents	2
3	案例一：Virtual Lab 如何设计 SARS-CoV-2 nanobodies	3
4	案例二：Paper2Agent，把论文变成可交互的研究助手	4
5	这类系统到底解决了什么问题	5
6	从 Virtual Lab 到 Agents for Science 的生态	6
7	源缺口说明	7
7.1	本章小结	7
8	总结与延伸	7

1 为什么科学发现会成为 agentic AI 的高价值场景

James Zou 这一讲的起点非常清晰：科学研究本身就是一个闭环过程，包含假设提出、文献检索、实验设计、工具调用、结果分析、复验与迭代。LLM agent 之所以适合科学发现，不是因为它们天然“聪明”，而是因为科学工作本来就高度依赖结构化协作、工具链调用和长时间尺度上的累积。

从课程视频和官方 readings 的共同信息来看，最有前景的路径不是让单个大模型包办全部科研工作，而是让多个角色化 agent 共同参与：有人负责方向判断，有人负责文献浏览，有人负责实验设计，有人负责数据分析，还有人负责挑错与复核。人类研究者则继续承担高层约束、伦理边界、方向筛选和最终判断。

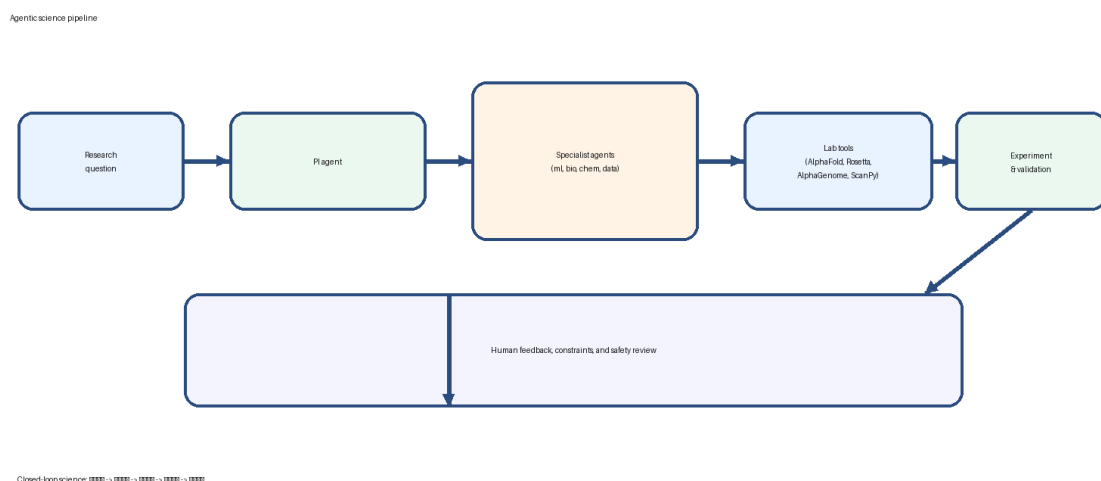


图 1: 本讲的总括图：科学发现可以被看作“目标定义 -> 专家分工 -> 工具执行 -> 结果验证 -> 人类反馈”的闭环系统。

2 Virtual Lab：把研究组织成一组可协作的 agents

课程中最有代表性的系统是 Virtual Lab。它不是一个单一聊天助手，而是一个由多个 LLM scientist agents 构成的研究组织。核心角色包括：

- **PI agent**：像 principal investigator 一样负责目标分配、课题筛选和会议调度。
- **student agents**：承担具体子任务，例如文献阅读、实验方案草拟、数据处理或工具调用。
- **teacher agents**：提供更系统化的训练任务和反馈，让 student agents 学会特定技能。
- **critic agent**：专门挑错、找漏洞、提出替代解释。
- **memory / sandbox**：让 agents 能记录上下文、保存中间结果，并在受控环境里调用外部工具。

这套组织的关键不是“每个 agent 都很强”，而是它们在制度上分工明确。PI agent 不需要自己知道所有细节；它只要知道何时该调用什么专家、何时该做汇总、何时该回到人类确认即可。教师 agent 的作用是把研究任务拆成训练问题；critic agent 的作用是防止系统在长链路里漂移。课程强调，这类结构特别适合科学任务，因为科学本身就是跨领域协作，而不是单一推理轨迹。

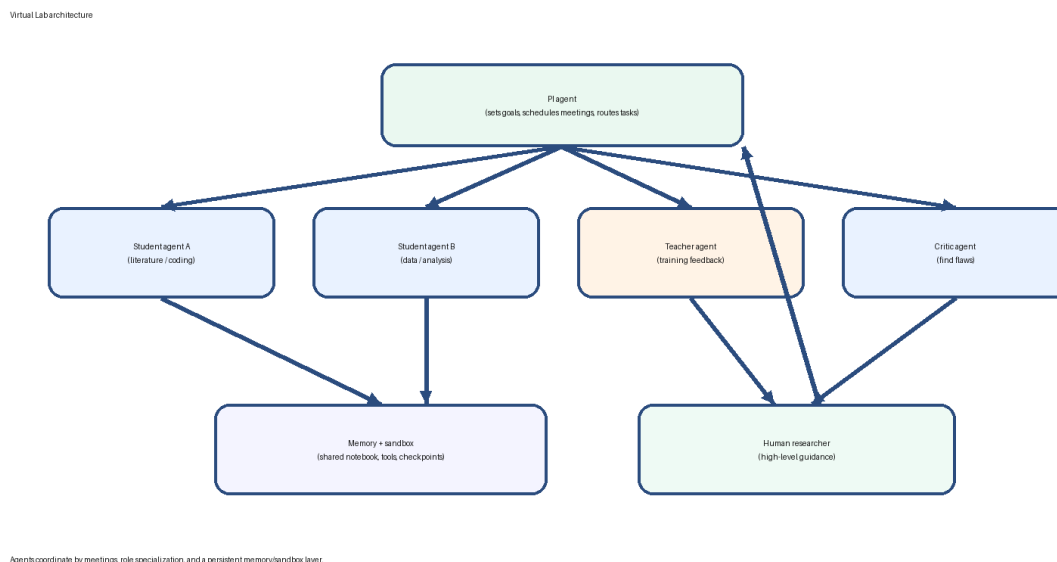


图 2: Virtual Lab 的组织方式: PI agent 统筹, 专长 agent 执行, critic agent 挑错, memory 与 sandbox 维持长期研究状态。

3 案例一: Virtual Lab 如何设计 SARS-CoV-2 nanobodies

官方 reading 《The Virtual Lab of AI agents designs new SARS-CoV-2 nanobodies》给出了这套方法最有说服力的例子。系统不是凭空“生成一个分子”，而是沿着科学上真实可操作的管线推进。大致流程是：

1. 设定研究目标：面向最近变异株设计可结合的 nanobody binders。
2. 由 agent 生成候选序列，并对候选进行筛选。
3. 用 AlphaFold-Multimer、Rosetta 和相关蛋白语言模型对结构与结合可能性做评估。
4. 将保留下来的候选送往实验验证，观察其对不同变体的 binding profile。
5. 结合实验结果继续修正筛选标准与设计策略。

论文摘要明确给出几个关键事实：系统最终设计出 92 个新的 nanobodies；实验验证显示其中一些分子对 JN.1 或 KP.3 变体有更好的结合，同时仍保留对祖先 spike protein 的强结合；这说明 AI agents 不只是能“写计划”，而是能在真实实验闭环里推动发现。

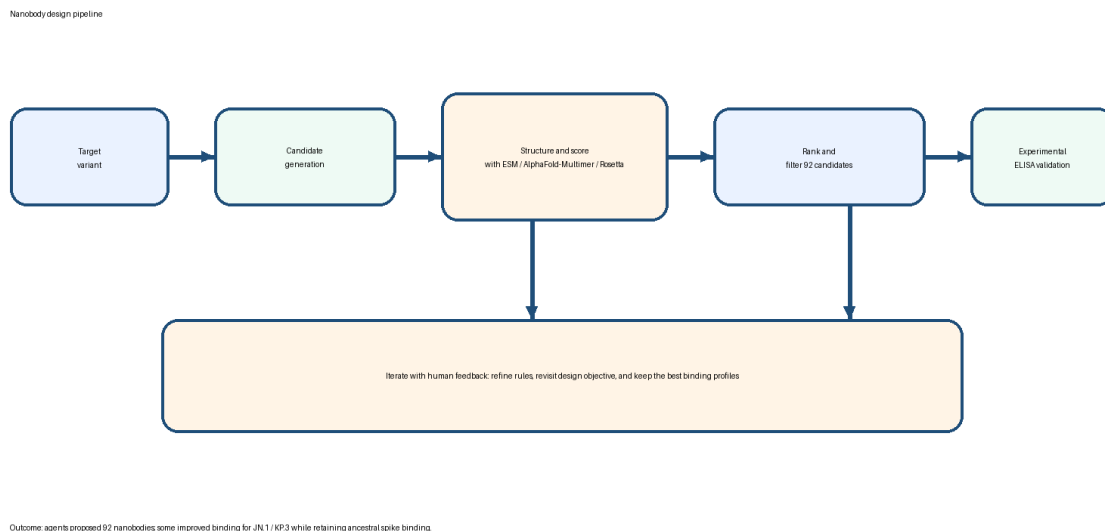


图 3: Nanobody 设计的 agentic pipeline: 从目标定义、候选生成、结构评估到实验验证。

这一案例的重要性在于它展示了一种现实主义的自动化科学观: AI 并没有取消实验, 也没有取消人类专家, 而是把研究流程拆得更细, 并把每一步的中间产物变成可复用的上下文。换句话说, agent 的价值不是替代实验, 而是让实验之前的迭代成本大幅下降。

4 案例二: Paper2Agent, 把论文变成可交互的研究助手

第二篇官方 reading 《Paper2Agent: Reimagining Research Papers As Interactive and Reliable AI Agents》把课程主题推进了一步: 不仅科学实验可以 agent 化, 研究论文本身也可以被转化为 agent。这里的核心思想是把静态论文变成一个能回答问题、能调用工具、能复现结果的交互式系统。

Paper2Agent 的工作流非常清晰:

- 多个 agent 分别分析论文文本和对应 codebase。
- 系统据此构建一个 Model Context Protocol (MCP) server, 把论文的方法、数据与工具包装成可调用接口。
- 随后通过迭代测试不断修复和加固这个 paper agent, 直到它既能复现论文结果, 又能回答新的科研查询。
- 最后把该 MCP 接到聊天式 agent 上, 例如 Claude Code, 形成面向研究者的自然语言接口。

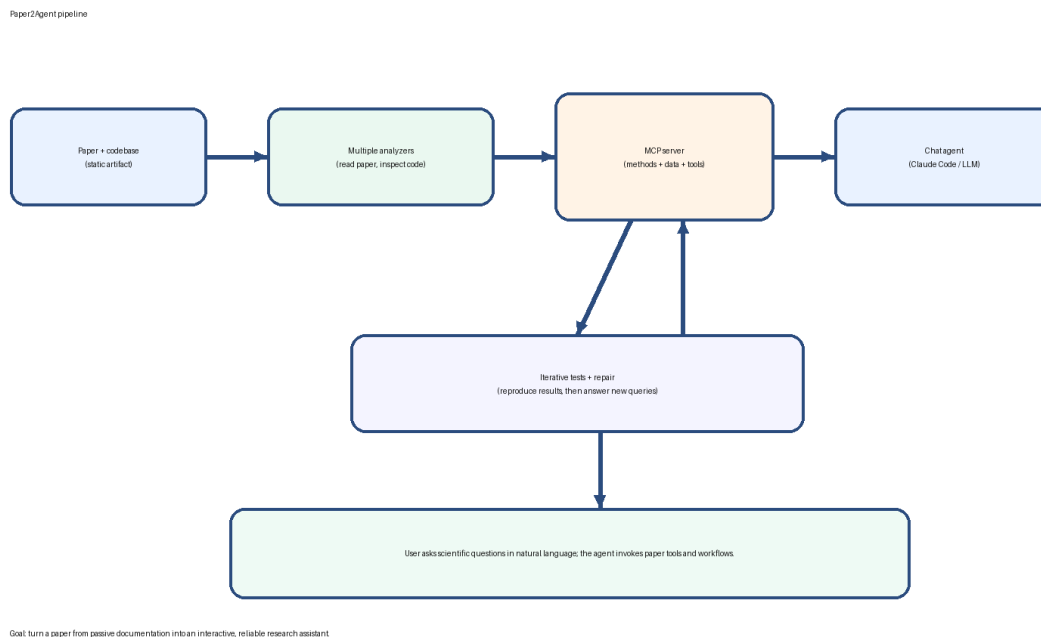


图 4: Paper2Agent 的核心思想：把论文、代码和数据封装成一个可交互的 MCP 研究代理。

论文摘要中提到的案例包括 AlphaGenome、ScanPy 和 TISSUE。系统利用这些基础工具构建出分别用于基因变异解释、单细胞分析和空间转录组分析的 paper agents。课程视频里还特别提到，这类系统可以让两个 paper agent 直接进行 agent-to-agent collaboration，例如一个方法 agent 与一个数据 agent 直接协作，从而把跨论文的交互变成研究常态。

5 这类系统到底解决了什么问题

这讲不是在鼓吹“全自动科学”，而是在总结几个当前最现实的增益点。

1. **把知识从静态文档变成可执行接口**：研究者不必手工翻论文、找代码入口、猜参数。
2. **把长链路研究拆成可审计步骤**：每个 agent 的输出都可以被记录、回放和复核。
3. **把专业分工显式化**：不同 agent 负责不同学科、不同工具和不同层次的判断。
4. **把人类从繁琐执行中解放出来**：人类更适合定义目标、检查结果和做最终判断。

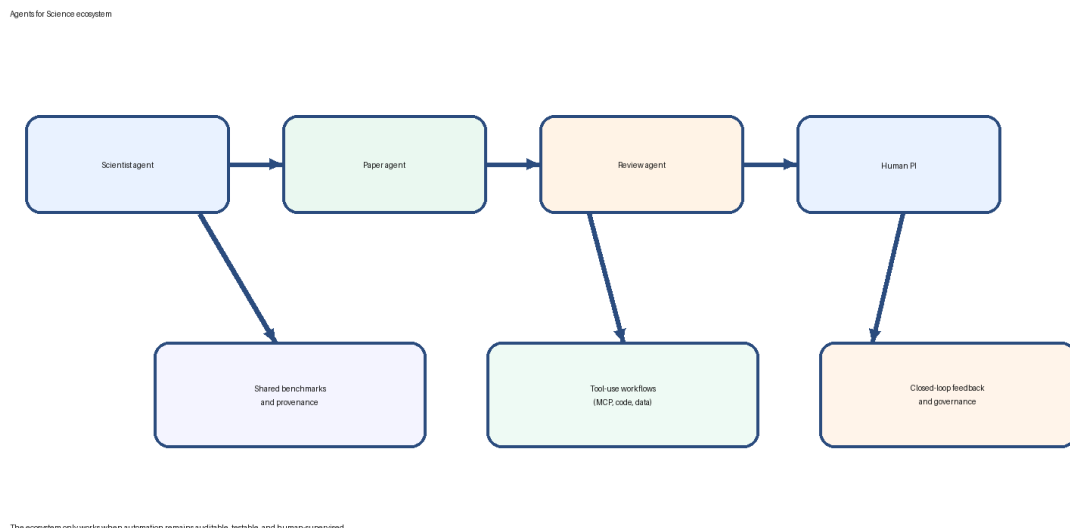


图 5: Agentic science ecosystem 的直观图景：paper agent、review agent、scientist agent 与人类研究者共同构成协作网络。

但课程同样强调了边界。第一，agent 仍然会 hallucinate，尤其是在长链路科研任务里。第二，很多研究工具并不是一开始就适合直接被“灌进模型”，因为文档、代码和数据结构可能过于复杂。第三，科学本身需要真实的闭环验证，不能只靠语言模型自说自话。Paper2Agent 对这些问题的回应不是回到纯手工，而是通过测试、迭代和工具封装把风险局部化。

6 从 Virtual Lab 到 Agents for Science 的生态

课程最后把视野放宽到一个更大的生态系统：Agents for Science。这里的意思不是单个论文，而是一个正在成形的研究共同体，包括自动科研、论文评审 agent、科学问答 agent、实验规划 agent 和人机协作工作流。课程视频提到，相关领域已经开始组织专门会议，说明这是一个快速增长的研究方向，而不是单点 demo。

在这个生态里，最关键的设计原则不是“放开自动化”，而是“逐步提升自主性”。合理的路径通常是：

- 先让 agent 做文献检索、总结和工具调用。
- 再让它做局部实验设计、候选筛选和结果分析。
- 再通过专项 benchmark 和人类反馈校准它的可靠性。
- 最后才讨论更高层次的自治和跨 agent 协作。

7 源缺口说明

这一讲没有官方 slide PDF。讲义因此采用了视频字幕、课程页以及两篇官方 readings 作为主来源，并用自绘结构图补足教学表达。这个缺口是公开资源层面的事实，不是讲义写作失败；它只意味着这章的可视化更依赖教材化重构，而不是逐页复刻 slides。

7.1 本章小结

这一讲的核心结论很明确：AI agents 在科学发现中的价值，不在于替代研究者，而在于把科学流程拆成可交互、可审计、可复用的模块。Virtual Lab 说明了多 agent 分工如何嵌入科学组织；nanobody case study 说明了 agent 真的可以进入实验闭环；Paper2Agent 则进一步说明论文本身也可以成为 agent 的载体。三者合起来，构成了“科学知识从静态对象变成动态系统”的路线图。

8 总结与延伸

如果把这一讲和前两讲放在一起，就会发现一个共同结构：无论是多智能体博弈、评测噪声还是科学发现，真正重要的都不是单点模型能力，而是交互结构、反馈回路和制度化协作。Agentic AI 最有前景的方向，正是把这些结构变成可工程化、可复现、可审计的系统。

后续如果继续深入，最值得扩展的方向有两个：一是如何设计更可靠的 paper agents，让它们在不牺牲 provenance 的前提下支持自然语言查询；二是如何把实验室里的多智能体协作规范化为更通用的科学工作流。前者解决“怎么用”，后者解决“怎么扩展”。

10 Practical Lessons from Deploying Real-World AI Agents

课程讲义

CS294/194-196 第 10 讲：现实世界 AI Agents 的部署经验

Codex 基于 Clay Bavor 讲座视频、官方 slides 与补充 reading 重写

2026-04-16



UC Berkeley Center for Responsible,
Decentralized Intelligence

AGENTIC AI MOOC
PRACTICAL LESSONS
FROM DEPLOYING REAL-
WORLD AI AGENTS
NOVEMBER 10, 2025

Clay Bavor
Co-founder


课程：UCB CS294/194-196 Agentic AI (Fall 2025)

讲次：Practical Lessons from Deploying Real-World AI Agents

主讲：Clay Bavor, Sierra

视频：<https://www.youtube.com/watch?v=sfJM4LaiYsM>

目录

1	为什么“部署经验”值得单独讲一讲	2
1.1	本章小结	2
2	从互联网到 mobile，再到 agent：交互范式为什么会再换一次	2
2.1	为什么 agent 更适合围绕任务而不是页面设计	3
2.2	本章小结	3
3	Conversation 是接口，但 workflow 才是产品	3
3.1	为什么企业 agent 需要比 chatbot 更强的状态管理	4
3.2	本章小结	4
4	什么叫“部署成功”：可靠性、评测与 Voice Sims	4
4.1	“Trust”不是抽象美德，而是系统属性	4
4.2	本章小结	4
5	从企业角度看 agent：不是 demo，而是业务操作系统的一部分	5
5.1	本章小结	5
6	总结与延伸	5
6.1	拓展阅读	5
6.2	本章小结	5

1 为什么“部署经验”值得单独讲一讲

前面的几讲主要在讨论 agent 的训练、评测、后训练和系统设计，而 Clay Bavor 这一讲把焦点直接拉到一个更现实的问题上：当 agent 真正面对企业客户、真实用户和客服 workflows 时，什么才算“做对了”？这是一种非常重要的课程转向，因为它把我们 from 研究性指标拉回到了生产系统的成功标准。

这讲的基调并不是“再讲一遍模型能力”，而是强调：现实世界 agent 的价值，往往不来自单个 model capability，而来自整个体验是否围绕 job-to-be-done 重构。也就是说，模型越强，越不能只把它塞进旧交互壳子里。

1.1 本章小结

本讲的价值在于把 agent 从“研究对象”转成“交付对象”。当系统面对客户体验、业务指标和真实故障时，设计原则会发生明显变化。

2 从互联网到 mobile，再到 agent：交互范式为什么会再换一次

官方 slides 里最醒目的一组类比，是“Internet → Websites”、“Mobile → Apps”、“AI → Agents”。Clay Bavor 想表达的不是一句营销口号，而是一个交互范式的变化：如果 AI 真正成为主接口，那么应用层的组织方式就不该只是在原有网页或 App 上加一个聊天框，而应当围绕 agent 的任务执行逻辑重新设计。

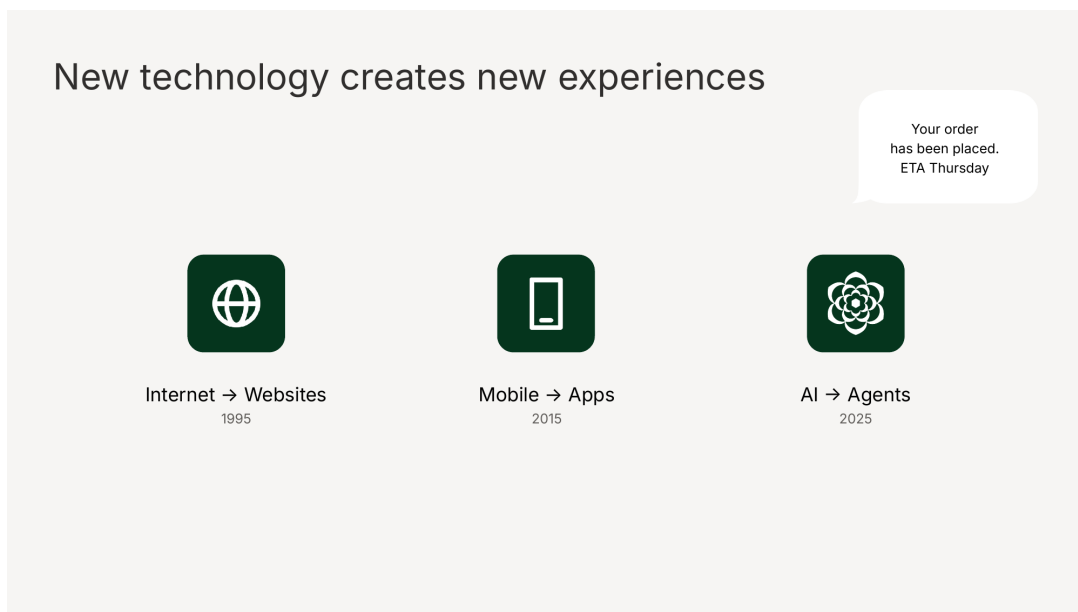


图 1: 技术浪潮与交互形态的对应关系：网站、App 与 agent 并不是同一层抽象的简单替换。

slides 紧接着又把这个判断落到一句很具体的话上：**The conversation is the interface.** 这句话的真正含义不是“未来所有产品都只剩聊天窗口”，而是说交互单元从静态页面和按钮，转向了围绕意图、上下文和任务结果持续展开的会话回路。对企业场景尤其如此，因为用户并不想“操作系统”，他们想“完成任务”。

2.1 为什么 agent 更适合围绕任务而不是页面设计

当用户面对一个客服、销售或支持场景时，他们关心的不是系统内部做了多少步推理，而是问题有没有被解决。因此 agent 最自然的产品抽象，不是“一个更会说话的 UI”，而是“一个能把任务持续推进到完成的执行体”。这也是 slides 里 ‘Pay for a job well done’ 那页真正强调的重点：计费、价值评估和系统设计都应该围绕任务完成度，而不是 token 数或交互次数。

部署视角下的核心判断

现实世界 agent 的接口不是 model API，而是任务结果。对企业客户而言，最重要的问题永远是：问题是否被解决、是否可靠、是否可持续扩展。

2.2 本章小结

从网站到 App，再到 agent，本质上是系统和用户之间的契约在变化。agent 的出现要求我们围绕 intent、context 和 task completion 去重写交互逻辑。

3 Conversation 是接口，但 workflow 才是产品

如果把这一讲压缩成一句最像教材的判断，那就是：**conversation 只是表层接口，真正决定产品价值的是隐藏在后面的 workflow 设计**。Clay Bavor 花了不少篇幅说明，企业 agent 不是一轮问答，而是包含信息收集、身份确认、知识检索、工具调用、状态更新、handoff 和结果校验的一整条链。

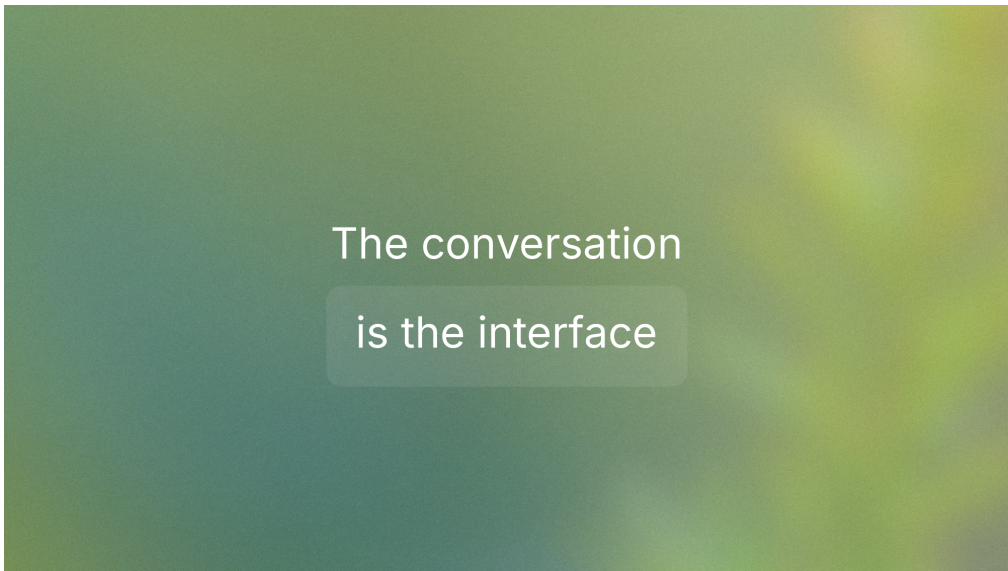


图 2: “The conversation is the interface” 的正确理解：对用户可见的是对话，对系统真正关键的是后面的状态流与任务流。

这件事和通用聊天产品有本质差异。对通用聊天产品来说，用户容忍一定程度的 exploratory interaction；但在企业流程里，客户往往只关心某件事是否被可靠完成，例如订单状态查询、账户修改、退款、预约或升级服务。如果系统不能稳定接住这些流程，它就算再会聊天，也只是演示系统。

3.1 为什么企业 agent 需要比 chatbot 更强的状态管理

企业 agent 几乎天然需要持久状态和结构化上下文。一次客服对话可能跨越多个回合，涉及账户信息、历史订单、政策规则、工具返回结果和升级工单状态。没有这些结构化状态，系统就很容易在多轮过程中自相矛盾。也正因如此，这一讲虽然没有用很重的系统术语，但它实际上一直在强调 memory、policy、tool use 和 workflow orchestration 的重要性。

3.2 本章小结

现实世界里的 conversation 不是自由聊天，而是执行入口。真正的系统设计单位是 workflow，而不是单轮 prompt。

4 什么叫“部署成功”：可靠性、评测与 Voice Sims

这讲最有工程价值的部分之一，是它没有把部署成功等同于“内部 demo 看起来很聪明”。相反，它把问题转向：**在真实环境里，如何评价 agent 是否真的可靠？**这正是补充阅读‘2-Bench’和‘Voice Sims’出现的原因。

‘2-Bench: Evaluating Conversational Agents in a Dual-Control Environment’的意义在于，它把对话 agent 放回一个更真实的评测框架：agent 不是在静态 benchmark 上答题，而是在会话与环境共同演化的条件下执行任务。对 agent 研究来说，这比单纯的 closed-form QA benchmark 更接近真实世界，因为很多错误都发生在状态切换、策略执行和用户交互细节里。

‘Voice Sims’则代表另一类更偏产品与运营的评测方式：在真正面向客户之前，用接近真实世界条件的模拟交互去测试 agent。它的核心价值不是“多跑几个 scripted test”，而是提前暴露流程卡点、边界条件、对话崩坏和 handoff 失败。换句话说，这类评测是在告诉我们：**agent 的可靠性要在 deployment loop 里被验证，而不是只在研究论文里被宣称。**

为什么评测在部署阶段反而更重要

研究阶段的评测帮助你比较方法；部署阶段的评测帮助你控制风险。后者往往更依赖真实 workflow、真实策略约束和真实用户行为分布。

4.1 “Trust”不是抽象美德，而是系统属性

Sierra 的企业定位使得这讲反复回到 trust。这里的 trust 并不只是一个品牌词，而是由一组具体系统属性组成：可预测的行为边界、正确的策略执行、稳定的 handoff、可靠的身份与权限控制，以及对失败模式的及时暴露和修复。换句话说，agent 一旦部署到 customer-facing 流程里，trust 就等于系统工程。

4.2 本章小结

部署成功不等于“模型回答得像人”，而等于“系统在真实任务里可评测、可约束、可纠错”。这也是 agent 走向生产的门槛。

5 从企业角度看 agent：不是 demo，而是业务操作系统的一部分

Clay Bavor 这讲还有一个很重要的视角转换：企业购买的不是一个“炫酷 demo”，而是一种新的 customer operation layer。agent 若真要落地，就必须接入企业已有的政策、数据、CRM、知识库、工具权限和人工团队。也就是说，agent 不是替代企业流程，而是嵌入并重构企业流程。

这一点对理解 agent 基础设施很重要。企业 agent 至少需要：

- 明确的 policy boundary；
- 对工具调用和数据访问的可控权限；
- 失败后的人工接管和审计轨迹；
- 能够根据真实反馈持续迭代的 evaluation loop。

这也是为什么这讲虽然表面上更像产品课，但它与前面的 systems 和 evaluation 章节高度一致。它只是在用企业落地语言复述同一个结论：agent 不是孤立模型，而是一个要进入复杂组织流程的系统。

最容易被忽略的部署风险

如果把企业 agent 理解成“客服场景里的聊天模型”，就会低估真实部署难度。真正的难点在于 policy、workflow、handoff、权限和评测闭环，而不是单次回复的流畅度。

5.1 本章小结

企业场景让 agent 的系统属性彻底暴露出来：你不仅要会生成答案，还要会完成流程、遵守政策、接受评测并在失败时回退。

6 总结与延伸

这讲最值得带走的结论有四点。

第一，agent 代表的是新的交互范式，而不是旧界面上的 AI 插件。

第二，conversation 是表面接口，workflow 才是真正产品。

第三，现实世界部署的核心不只是更强模型，而是可靠性、可评测性与 trust。

第四，企业 agent 的成功标准是 job-to-be-done 是否被稳定完成，而不是对话是否“像人”。

6.1 拓展阅读

- τ^2 -Bench: Evaluating Conversational Agents in a Dual-Control Environment
- Voice Sims: test agents in real world conditions before they talk to customers

6.2 本章小结

这一讲把 Agentic AI 从研究原型拉到了真实企业流程里。读完之后，应该更清楚地看到：部署 agent 的难点在系统工程，而不在一句更聪明的 prompt。

11 Multi-Agent Systems in the Era of LLMs Oriol Vinyals,

课程讲义

CS294/194-196 第 11 讲：LLM 时代的多智能体系统

Codex 基于 Oriol Vinyals 讲座视频与官方课程页重写

2026-04-16



UC Berkeley Center for Responsible,
Decentralized Intelligence

AGENTIC AI MOOC

ALPHASTAR REVISITED:
MULTI-AGENT SYSTEMS
IN THE ERA OF LLMS

NOVEMBER 17, 2025



Oriol Vinyals
VP, Research


课程：UCB CS294/194-196 Agentic AI (Fall 2025)

讲次：Multi-Agent Systems in the Era of LLMs

主讲：Oriol Vinyals, Google DeepMind

说明：官方课程页未提供 slides，本章依据视频、字幕和课程页做 best effort 教材化重建。

视频：<https://www.youtube.com/watch?v=ntj0xjZMaac&t=2s>

目录

1 为什么这讲要回头讲 AlphaStar	2
1.1 本章小结	2
2 什么叫 agent：从 reward maximizer 到用户驱动的复合系统	2
2.1 为什么 “user 也在 loop 里” 会改变一切	3
2.2 本章小结	3
3 多智能体为什么会再次重要	3
3.1 多智能体不只是 “并行”，而是结构化分工	3
3.2 本章小结	4
4 从游戏环境到开放 agent harness	4
4.1 为什么这和 agent engineering 直接相关	4
4.2 本章小结	4
5 对 LLM 时代的具体启示：哪些 old lessons 仍然成立	4
5.1 本章小结	5
6 总结与延伸	5
6.1 本章小结	5

1 为什么这讲要回头讲 AlphaStar

Oriol Vinyals 一开场就把讲座分成两部分：一部分回顾 AlphaStar 那类传统强化学习（reinforcement learning）智能体系统，一部分把这些经验重新映射到今天的 Gemini 与 LLM agents。这个组织方式非常重要，因为它说明：**多智能体系统并不是 LLM 时代凭空冒出来的新话题，而是许多老问题在新接口上的重现。**

讲者强调 “ideas recirculate”。这句话的意思不是历史简单重复，而是说：很多我们今天在 LLM agent 里重新遇到的问题，例如环境定义、目标函数、工具接口、分工、协作与评估，其实早在游戏 AI 和 multi-agent reinforcement learning 里就已经被系统研究过。区别在于，今天的 environment 更开放、更模糊，用户也成了环路的一部分。

1.1 本章小结

这讲的起点不是“多加几个 agent 就更强”，而是把多智能体放回 agent research 的长期历史里，先搞清楚它究竟继承了哪些旧问题。

2 什么叫 agent：从 reward maximizer 到用户驱动的复合系统

讲者在 transcript 里非常明确地重述了一个经典定义：agent 有 goals、observations、actions，并在 environment 中循环优化 reward。这个定义来自更传统的 AI / RL 视角。在单环境、规则明确的场景里，它足够强大，也很清晰。

但讲者马上指出，今天的 agent 已经不止是“连到一个 environment 里最大化 reward”的对象。用户、聊天界面、搜索、terminal command、浏览器、外部工具和长程状态，都让 environment 本身变得更丰富，也更不封闭。于是 agent 的定义从一个简洁的 RL loop，扩展成一个用户、模型、工具与环境共同作用的复合系统。

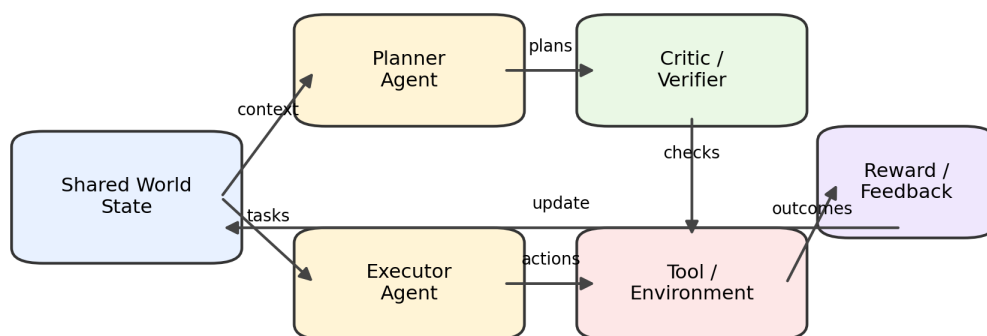


图 1: 从 shared world state、planner、executor、critic 到 tool/environment 的协作框架。它并不是某个特定系统的源码图，而是对本讲多智能体协调逻辑的教学化重述。

这讲对 agent 定义的关键变化

在传统 RL 里，环境、观测、动作和奖励往往边界清晰；在 LLM 时代，用户本身也进入了环境，而 environment 又进一步扩展成搜索、终端、浏览器、API 和人类反馈共同构成的开放系统。

2.1 为什么“user 也在 loop 里”会改变一切

一旦 agent 要服务真实用户，目标就不再只有一个显式 reward。很多时候，用户满意度、任务完成率、安全边界、策略遵守和资源消耗都要一起考虑。这让 multi-agent system 不只是“more agents”，而是需要重新定义 coordination objective：不同子 agent 为谁工作、用什么共享状态、何时终止、何时求助用户、何时切换工具。

2.2 本章小结

LLM 时代的 agent 不是把传统 RL agent 套个聊天接口，而是把用户、工具和环境都纳入了同一循环，因此系统复杂度显著上升。

3 多智能体为什么会再次重要

如果单个 agent 已经能调用工具和规划，为什么还需要 multiple agents？这讲给出的答案，不是“因为多个 agent 更炫”，而是因为任务本身越来越复杂，很多时候自然会分解成不同角色：规划、执行、批判、验证、工具操作、环境交互。把这些功能塞进一个 monolithic policy 里不一定最稳，也不一定最可解释。

从 AlphaStar 的角度，多智能体学习（multi-agent learning）首先出现在对手、队友、协作和竞争并存的环境里。讲者提到 curriculum of difficulty、不同游戏阶段的协同，以及在复杂环境中学到的 coordination lesson。这些经验迁移到 LLM 时代后，对应的就是：是否要有 planner agent、executor agent、critic agent，是否要让不同 agent 专注不同上下文，是否要通过 self-play 或 simulated interaction 获得更强策略。

3.1 多智能体不只是“并行”，而是结构化分工

本讲隐含的一个重要思想是，多智能体的价值不在于多开几个线程，而在于结构化分工。不同 agent 可以拥有不同的观察窗口、工具权限、验证职责和记忆范围。这样设计的好处有三层：

- **降低单体复杂度**：一个 agent 不必在同一时刻既规划、又执行、又审查自己。
- **增强可解释性**：失败时更容易定位是 planner、executor 还是 critic 的问题。
- **更贴近组织现实**：现实 workflow 本来就是角色分工，而不是一个全能体包办一切。

多智能体的真正收益

multi-agent 的真正收益不是“数学上 agent 数量更多”，而是把复杂任务拆成了更接近组织结构和工具权限边界的子系统。

3.2 本章小结

多智能体的重要性来自任务结构，而不是技术时髦。系统之所以拆分成多个 agent，是因为开放环境里的目标、工具和责任本来就更适合分工。

4 从游戏环境到开放 agent harness

讲者反复回到一个对比：过去的环境是 clearly defined single environment，现在的 agent harness 则更一般，可能包含 search、terminal、browser、external APIs，甚至用户反馈本身。这一转变意味着 multi-agent systems 的难点也变了。

在游戏环境里，很多状态与奖励可以精确定义；而在开放 harness 里，状态空间更加异质，行动效果也更延迟、更难验证。比如一个 agent 可以发起搜索、读取网页、写入文件、调用终端命令，每一步都可能改变后续可见世界。此时 multi-agent coordination 不能只靠一个简单 reward signal，而需要更强的状态共享、日志记录、失败恢复和外部评测。

4.1 为什么这和 agent engineering 直接相关

这讲虽然来自 DeepMind 的视角，但它对今天 agent engineering 的启示非常直接：如果你希望多个 agent 协作完成任务，系统至少要有一个足够明确的共享状态表示。没有 shared state，多 agent 很容易沦为重复劳动或互相覆盖；没有明确的终止条件，多 agent 又会陷入空转和无限争论。

多智能体最容易被误解的地方

如果没有共享状态、边界清晰的职责和终止机制，多智能体系统很容易比单 agent 更差。多智能体不是免费午餐，它首先是一种更复杂的系统工程。

4.2 本章小结

开放 agent harness 让多智能体重新变得有价值，但同时也让 coordination 的要求更高。系统设计要解决的，不只是“谁来做”，更是“谁知道什么、谁更新什么、谁来停下”。

5 对 LLM 时代的具体启示：哪些 old lessons 仍然成立

讲者之所以把 AlphaStar 和 Gemini / LLM agents 放在同一讲里，不是为了做 nostalgia，而是为了提出一个判断：很多今天被重新发明的技巧，本质上早就以另一种形式出现过。最值得保留的 lessons 至少有三条。

第一，**环境抽象必须足够清晰**。无论是游戏还是浏览器、终端、搜索 API，如果 agent 不能形成稳定的 environment model，就很难做可靠的长期规划。

第二，**coordination 需要显式结构**。不管是 self-play、对手建模，还是 planner / executor / critic 分工，本质上都是在避免“所有行为混成一个黑箱”。

第三，**evaluation 必须贴近真实任务**。在游戏里，胜率和奖励很明确；在 LLM agents 里，替代它们的应该是 task completion、correctness、tool-usage quality 和 user satisfaction，而不是只看表面流畅度。

5.1 本章小结

这讲对 LLM 时代最重要的启示，是不要把多智能体看成全新的 buzzword，而要把它看成旧的 agent ideas 在新环境里的再组织。

6 总结与延伸

这讲最值得带走的结论有四点。

第一，多智能体不是 LLM 时代突然出现的新概念，它延续了长期的 agent / RL 研究传统。

第二，今天 agent 的 environment 比过去更开放，也更接近真实用户和真实工具，因此 coordination 问题反而更难。

第三，multi-agent 的价值在结构化分工，而不是 agent 数量本身。

第四，很多所谓“今天的新问题”，其实都可以从 AlphaStar 一类系统里找到结构前身。

6.1 本章小结

Lecture 11 的真正价值，是把多智能体从时髦标签重新变回系统问题：什么是环境，什么是状态，谁来规划，谁来执行，谁来评估，以及这一切如何形成闭环。

12 Autonomous Agents: Embodiment, Interaction, and Learning

课程讲义

CS294/194-196 第 12 讲：具身自治体、交互 与学习

Codex 基于 Peter Stone 讲座视频与官方 readings 重写

2026-04-16



UC Berkeley Center for Responsible,
Decentralized Intelligence

AGENTIC AI MOOC
AUTONOMOUS AGENTS:
EMBODIMENT,
INTERACTION, AND
LEARNING
DECEMBER 1, 2025



Peter Stone
Chief Scientist at Sony AI,
Professor at UT Austin
Sony AI

课程：UCB CS294/194-196 Agentic AI (Fall 2025)

讲次：Autonomous Agents: Embodiment, Interaction, and Learning

主讲：Peter Stone, UT Austin / Sony AI

说明：官方课程页未提供 slides，本章依据视频、字幕与官方 readings 做 best effort 教材化重建。

视频：<https://www.youtube.com/watch?v=iDhzzugMOLA>

目录

1	为什么要把“embodiment”重新放回 agent 讨论中心	2
1.1	本章小结	2
2	什么叫 autonomous agent：传感、决策与执行的连续闭环	2
2.1	本章小结	3
3	从 robot soccer 到 humanoids：多智能体与具身学习的长期路线	3
3.1	为什么 humanoids 让问题进一步升级	3
3.2	本章小结	3
4	为什么 simulation 和 representation learning 变成关键	3
4.1	本章小结	4
5	LLM agent 课程为什么需要 robotics 这一讲	4
5.1	本章小结	4
6	总结与延伸	4
6.1	拓展阅读	5
6.2	本章小结	5

1 为什么要把“embodiment”重新放回 agent 讨论中心

Peter Stone 一开场就把这讲和前面几讲区分开了：课程前半段已经覆盖了 learning、reasoning、multi-agent 和 scientific discovery，但对 **embodiment** 的讨论还不够。这里的 embodiment 不是抽象的“把 agent 放到世界里”，而是非常具体地指向 robots，包括 physical robots 与 simulated robots。

这讲最重要的第一个判断是：**all robots are agents, but not all agents are robots**。这句话把 agent 与 robot 的关系说得很清楚。agent 是更一般的概念，只要系统通过 sensors 和 effectors 与环境形成持续闭环，它就是 agent；而 robot 则是在此基础上进一步要求与 physical world 发生真实交互的特例。

1.1 本章小结

把 embodiment 放回 agent 主线，意味着课程不再把 agent 仅仅理解成浏览器、终端和 API 上的数字执行体，而是把它扩展到物理环境中的自治系统。

2 什么叫 autonomous agent：传感、决策与执行的连续闭环

这讲给出的定义非常经典：an agent is an intelligent system that interacts with the environment through sensors and effectors。Peter Stone 特别强调 autonomous 的含义在于连续闭环，而不是一轮一轮被外部调用。换句话说，agent 不是“等一个 prompt 再给一个 response”，而是持续接收环境输入、形成内部状态、输出动作，并让这些动作影响下一轮感知。

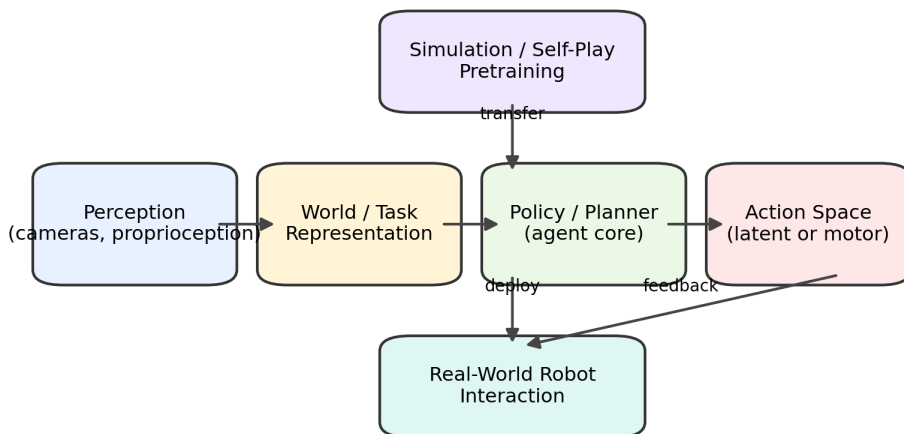


图 1: 具身自治体的最小闭环：感知、世界/任务表示、策略与规划、动作空间，以及 simulation-to-real 的迁移路径。

一旦把定义写成连续闭环，很多看似只是 robotics 里的问题，立刻就变成了 agent 的核心问题。例如：

- 环境观测是不完整且有噪声的；
- 行动会改变未来观测；
- 奖励、任务目标与安全约束常常不是一次性给定；

- 学习必须和 interaction 紧密耦合。

为什么 robotics 会让 agent 定义更清楚

在浏览器或代码环境里，agent 的 effectors 可能被误看成“只是 API 调用”；在机器人环境里，effectors 直接作用于物理世界，因此 perception–decision–action loop 的本质会被看得更清楚。

2.1 本章小结

这讲把 autonomous agent 定义重新校准成一个持续闭环系统。这种定义对数字 agent 和物理 robot 同样成立，只是后者把不确定性和安全代价放大了。

3 从 robot soccer 到 humanoids：多智能体与具身学习的长期路线

Peter Stone 明确把自己长期的研究线索摆出来：robot soccer、多机器人协作、reinforcement learning、再到 humanoid robots。这条线索的意义，不是罗列历史案例，而是说明具身 agent 的研究从来都不是单机控制问题，而是长期与多智能体、交互学习和真实系统约束绑在一起的。

RoboCup 之所以在这讲里占重要位置，是因为它同时包含 perception、decision making、multi-agent coordination 和 long-horizon task structure。它既是 robotics benchmark，也是 agent benchmark。讲者想借此说明：即便在 LLM 时代，多智能体与具身性也不是分离的话题，而是常常同时出现。

3.1 为什么 humanoids 让问题进一步升级

从 AIBO 到 humanoids，并不是“同样问题换个外形”，而是控制维度、感知难度、协作复杂度和安全风险都显著上升。humanoid robots 不仅要 do locomotion，还可能要 manipulation、navigation、human interaction 与长期任务执行。也因此，agent 的设计不再只是 policy 的事情，而是必须同时处理 action abstraction、state representation 和 real-world robustness。

3.2 本章小结

robot soccer 和 humanoids 说明，具身 agent 的难点是组合性的：感知、控制、协作和环境不确定性一起出现，不能被拆成互不相干的小问题。

4 为什么 simulation 和 representation learning 变成关键

这讲虽然没有官方 slides，但 transcript 和 readings 的组合给出了非常清晰的一条技术路线：**要做现实世界 robot agents，就必须把 simulation、representation learning 和 real-world adaptation 结合起来。**

补充 reading ‘Outracing Champion Gran Turismo Drivers with Deep Reinforcement Learning’ 强调的是大规模模拟环境、自博弈（self-play）和高度结构化的 reward / evaluation 如何推动复杂技能习得。它虽然是赛车场景，但对这门课的真正启示是：当 environment 足够 rich 且 evaluation 足够清晰时，agent 能通过长期 interaction 学出远超模仿学习的策略。

另一篇 reading ‘SLAC: Simulation-Pretrained Latent Action Space for Whole-Body Real-World RL’ 则把问题推进到更符合 2025 agent 叙事的方向：如何先在 simulation 中预训练 latent action space，再迁移到 real-world embodied RL。它的意义在于：**行动空间本身可以被学习和压缩。**对具身

agent 来说，这和 tool abstraction 在数字 agent 里的角色非常类似，都是在降低高维原始动作空间的复杂度。

这讲与前面 software / web agents 的对应关系

数字 agent 会抽象 tools 与 workflows，具身 agent 会抽象动作空间与交互接口。两者虽然环境不同，但核心工程动作相似：都在为复杂行动寻找更稳定、更可学习的中间表示。

4.1 本章小结

simulation、representation 与 action abstraction 并不是 robotics 里的配套细节，而是让具身 agent 真正可训练、可迁移、可部署的核心支柱。

5 LLM agent 课程为什么需要 robotics 这一讲

如果这门课只讲 software agents，很容易让人把 agent 理解成“会调工具的大模型”。Peter Stone 这讲的真正价值，是把 agent 概念重新拉回环境交互、长期学习与 embodied action 的一般形式。也就是说，LLM agent 只是 agent family 的一个分支，而不是定义本身。

从课程结构上看，这一讲至少补了三件事。第一，它提醒我们 memory 和 planning 不只出现在语言链条里，也出现在 physical interaction loop 里。第二，它让 evaluation 问题更难，因为 real-world task success 往往更昂贵、更慢、更难自动验证。第三，它把 safety 的 stakes 提前抬高：一旦动作进入 physical world，错误就不再只是错误文本，而可能是系统级事故。

为什么不能把具身 agent 当作“以后再说”的外延问题

如果 agent 课程完全忽略 embodiment，就会默认 environment 永远是可回滚、低成本、数字化的。这会让很多 planning、memory、evaluation 和 safety 讨论失真。

5.1 本章小结

这一讲把 agent 从语言和软件环境重新拓展到物理环境，迫使我们重新理解 autonomy、learning、evaluation 和 safety 的含义。

6 总结与延伸

这讲最值得带走的结论有四点。

第一，autonomous agent 的一般定义是持续的感知-决策-执行闭环，而不是单轮调用。

第二，robot 是具身 agent 的典型形态，它把 environment interaction 的本质放大得最清楚。

第三，simulation、representation learning 和 action abstraction 是具身 agent 的核心方法，而不是辅助技巧。

第四，robotics 之所以重要，不是因为它离 LLM 更远，而是因为它让我们更清楚地看到 agent 概念的真实边界。

6.1 拓展阅读

- *Outracing Champion Gran Turismo Drivers with Deep Reinforcement Learning*
- *SLAC: Simulation-Pretrained Latent Action Space for Whole-Body Real-World RL*

6.2 本章小结

Lecture 12 的意义在于把 agent 重新定义成一般性的自治交互系统。这样看完整门课时, software agents、scientific agents 与 embodied agents 才会被放进同一张地图里。

13 Agentic AI Safety & Security

课程讲义

CS294/194-196 第 13 讲: Agentic AI 的安全与安全性

Codex 基于 Dawn Song 直播录播字幕与官方课程页重写

2026-04-16



UC Berkeley Center for Responsible,
Decentralized Intelligence

AGENTIC AI MOOC

AGENTIC AI SAFETY &
SECURITY

DECEMBER 8, 2025



Dawn Song
Professor; Berkeley RDI
Co-Director
UC Berkeley

课程: UCB CS294/194-196 Agentic AI (Fall 2025)

讲次: Agentic AI Safety & Security

主讲: Dawn Song

说明: 本讲官方课程页没有提供 slides, 正文依据直播录播字幕与课程页信息做 best effort 教材化重建。

视频: <https://youtube.com/live/CvZDJxd4LKM?feature=share>

目录

1 为什么 agentic AI 的风险在 2025 年被重新放大	2
1.1 本章小结	2
2 Safety 与 Security: 为什么这两者既不同又分不开	2
2.1 本章小结	2
3 为什么 agent 系统的攻击面比单模型更大	2
3.1 本章小结	3
4 评测与风险评估: 没有 measurement 就没有安全	3
4.1 为什么 evaluation 会反过来塑造系统设计	4
4.2 本章小结	4
5 Defense 与 policy enforcement: 如何让 agent 在边界内行动	4
5.1 cyber security 为什么会成为最后的高风险场景	4
5.2 本章小结	4
6 总结与延伸	5
6.1 本章小结	5

1 为什么 agentic AI 的风险在 2025 年被重新放大

这一讲一开场就给出一个非常鲜明的张力：一方面，2025 被反复描述为 “year of agents”；另一方面，frontier AI 和 agentic AI 的快速增长也让安全（safety）与安全性（security）问题同步放大。Dawn Song 的核心判断是：一旦 **AI 系统能够在更广泛的环境里自主规划、调用工具、访问外部资源并执行动作**，攻击者就会比以往有更强的激励去利用这些系统，而系统失误的后果也会更严重。

她在开头引用了一份由众多学者参与的国际 AI safety 报告，其作用不是给出某个单点结论，而是说明风险谱系已经非常广，不再只是“模型会不会胡说八道”的层面。对 agentic AI 来说，风险问题从一开始就是系统问题，而不是最后再附加上的 compliance checklist。

1.1 本章小结

本讲的出发点不是保守主义，而是现实主义：agent 越强、越接近真实世界任务，攻击和误用的激励就越高，安全讨论也越不能停留在模型层表面。

2 Safety 与 Security：为什么这两者既不同又分不开

讲者在 transcript 里给出了一个很适合教材化的区分：**AI safety** 主要关心系统对外部环境可能造成的伤害，**AI security** 则主要关心系统本身如何抵御来自外部恶意主体的攻击与利用。这个区分很重要，因为它把两个常被混用的词分离开了。

如果把这两者写成更系统化的表述，可以理解为：

- **Safety**：系统是否会以错误、失控或不符合目标的方式作用于世界；
- **Security**：系统是否会被恶意输入、恶意环境、恶意工具或恶意用户所利用和操纵。

但讲者马上又补了一句更关键的话：它们在 agentic AI 上是分不开的。原因很简单，许多 safety mechanism 本身也必须在 adversarial setting 下成立。例如一个 alignment mechanism 如果在攻击者输入下轻易失效，那它就同时是 safety failure 和 security failure。

这讲最值得保留的定义

对 agentic AI 来说，safety 与 security 的关系不是两个平行模块，而是一个相互嵌套的系统问题：系统既要不伤害世界，也要不被恶意世界轻易接管。

2.1 本章小结

这讲没有把 safety 和 security 当成两个松散话题，而是明确指出：对 agentic system 来说，它们共享同一个闭环，只是关注方向不同。

3 为什么 agent 系统的攻击面比单模型更大

讲者接下来把注意力从 base model 拉到了整套 agent stack。她明确说，LLM 的安全与安全性固然是基础，但 agentic AI 的整体风险远不止模型层。原因在于 agent system 还会加入 perception、reasoning、planning、tool use、retrieval、memory 和 environment interaction。每增加一层能力，攻击面也就增加一层。

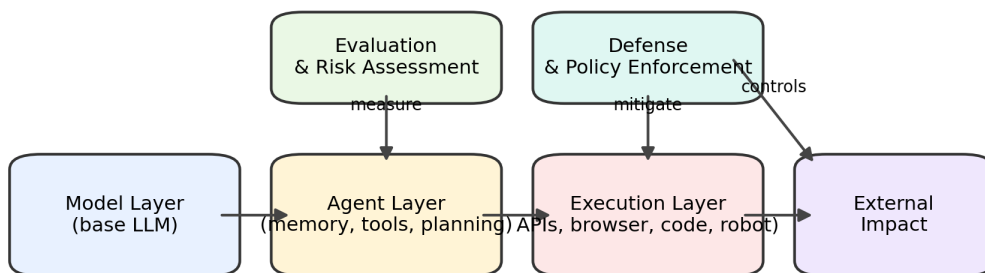


图 1: Agentic AI 的分层攻击面：从 base model，到 memory/tool/planning，再到 execution layer 与 external impact。

这讲对 agent stack 的直觉可以概括成：模型层的 prompt injection、越狱和 misalignment 只是基础风险；一旦系统允许调用工具、检索外部知识、维护 memory、执行 browser/terminal/robot actions，问题就会升级成完整的 system risk。例如：

- 工具调用可能被恶意输入诱导；
- retrieval 结果可能被投毒；
- memory 可能泄露、污染或被持久化利用；
- policy enforcement 若不明确，agent 可能越权执行。

为什么 agent 比单模型更脆弱

一个单纯生成文本的模型，失误通常停留在文本层；一个能调工具、记住历史、执行动作的 agent，失误会沿着 system interface 向外扩散。因此 agent 的风险管理必须从 system boundary 开始设计。

3.1 本章小结

agent 的能力增强并不是单调利好。memory、tools、retrieval 和 action 让系统更有用，也让系统更容易被攻击与误用。

4 评测与风险评估：没有 measurement 就没有安全

讲者在中段明确说，agentic AI 需要 evaluation 和 risk assessment。这个判断非常关键，因为它把安全讨论从抽象原则拉回到 operational discipline：如果没有稳定的 measurement，你就既无法比较风险，也无法验证 defense 是否有效。

对 agentic system 而言，evaluation 不能只在 model level 进行。至少需要有三层：

1. **Model-level evaluation**: base model 在安全、越狱、合规等方面的基础能力；

2. **System-level evaluation**: memory、tool use、policy、retrieval 和 workflow 组合后的整体行为;
3. **Adversarial / misuse evaluation**: 在攻击者存在的条件下, 系统会不会被操控、泄露信息或执行危险动作。

这也解释了为什么 transcript 后半段会反复提到 evaluation、risk assessment、trustworthiness evaluation 和 cyber security benchmark: agent 的安全不能靠“我们觉得它还行”, 而要靠明确的测量和持续的 red teaming。

4.1 为什么 evaluation 会反过来塑造系统设计

一旦你承认 system-level evaluation 必须存在, 就会倒逼系统增加更多可审计结构, 例如更清晰的 policy boundary、更可追踪的 action log、更明确的 memory state 和更严格的 tool permission。这一点和课程前面的 evaluation 课完全对齐: **evaluation 不是最后的打分动作, 而是系统设计本身的一部分。**

4.2 本章小结

agentic AI 的安全问题无法只靠原则陈述解决, 必须转化为可以持续测量、持续回归、持续对抗测试的评估对象。

5 Defense 与 policy enforcement: 如何让 agent 在边界内行动

讲者在安全栈里特别强调 defenses 与 policy enforcement。这个点很像 enterprise deployment 和 agent evaluation 课的延伸: 如果系统没有明确策略边界, 它再强也不安全; 如果系统有策略边界但不能执行, 它也不安全。

从 transcript 可抽象出一条相当明确的工程路线: 先定义允许与禁止的动作, 再把这些限制落实到 tool invocation、memory access、retrieval scope 和 execution environment 上, 最后再用 evaluation 去检验这些限制是否真的生效。也就是说, policy enforcement 不是独立组件, 而是嵌在 agent state update 与 action selection 过程里的约束系统。

这讲的 defense 直觉

最有效的防御通常不是给模型加一句“请安全回答”, 而是把 agent 能做什么、能看到什么、什么时候必须停下, 写进系统结构本身。

5.1 cyber security 为什么会成为最后的高风险场景

这讲结尾专门把 cyber security 单列出来, 不是偶然。对 agent 而言, cyber security 是最典型的高风险环境: 工具丰富、攻击者主动、反馈快速、潜在后果严重, 而且模型能力越强, 攻击者也越愿意把它当成 leverage。也正因如此, cyber security 对 agentic AI 来说既是应用机会, 也是风险放大器。

5.2 本章小结

defense 的核心不是抽象口号, 而是把策略约束写进系统执行路径。cyber security 则提醒我们: agent 能力一旦进入高对抗环境, 安全问题会迅速放大。

6 总结与延伸

这讲最值得带走的结论有四点。

第一，agentic AI 的风险问题不是 model-only，而是 full-stack system problem。

第二，safety 与 security 可以区分，但对 agent system 来说必须一起设计。

第三，memory、tool use、retrieval、planning 和 execution 会显著扩大攻击面。

第四，真正可靠的防御来自 evaluation、risk assessment 和 policy enforcement 的持续闭环，而不是一次性加固。

6.1 本章小结

Lecture 13 的意义，是把整门 Agentic AI 课程拉回一个现实边界：agent 越接近真实任务执行，越必须在系统层同时考虑安全、攻击、评估和治理。